

PROYECTO SEMESTRAL — ENTREGA 1

**Asignación de vueltas a buses de una misma ruta con
ventanas de inicio y tiempos de viaje dependientes del
tráfico**

*Heurísticas List Scheduling y LPT para máquinas idénticas, con Ramificación y Poda y
CP-SAT como referencia exacta, aplicadas al transporte urbano de Cusco*

Asignatura	Algoritmos Avanzados
Docente	Héctor Eduardo Ugarte Rojas
Grupo	Grupo 4
Tema asignado	Optimización de asignación de trabajos en máquinas idénticas
Atributo del graduado	AG_C06 – Aprendizaje a lo largo de la vida
Hito	Entrega 1 – Cierre del primer periodo

Integrantes	Código
Aguilar Quispe, Yon	111276
Choque Mamani, Brayan Isau	215719
Huacani de la Cruz, Dany	081561
Moreano Villena, Miguel Angel	211859

Resumen

Este informe corresponde a la Entrega 1 del proyecto semestral del Grupo 4, cuyo tema asignado es la **optimización de la asignación de trabajos en máquinas idénticas**. El tema se aplica a las 37 empresas/rutas de transporte urbano de Cusco registradas en la tabla de flota operativa de 2020: cada **bus** es una máquina, cada **vuelta** origen–destino–origen es un trabajo y cada ruta se resuelve como una instancia independiente.

Para responder a la observación del docente sobre el tráfico, la formulación se reestructuró. La duración de una vuelta ya no es constante: depende de su hora de salida mediante un perfil de congestión por franjas horarias (modelo de velocidad constante por tramos de Ichoua, Gendreau y Potvin, que garantiza la propiedad FIFO). A esa capa determinística se suma una capa estocástica que se evalúa por simulación Monte Carlo. Se definieron tres escenarios: T0 (sin tráfico, equivalente al Avance 1), T1 (perfil normalizado, media 1) y T2 (perfil pesimista). Además, el almuerzo de 2 h pasó a modelarse con una ventana de inicio, el objetivo se corrigió a la **carga máxima por bus $L_{\max}$** y se adoptó un criterio lexicográfico (primero vueltas atendidas, luego $L_{\max}$).

El prototipo, escrito en Python, implementa List Scheduling (LS) y LPT adaptados a ventanas, almuerzo y tráfico; un algoritmo de Ramificación y Poda propio; un modelo exacto en OR-Tools CP-SAT; un validador independiente; métricas; simulación y una interfaz de línea de comandos. Cuenta con 35 pruebas automatizadas. Los resultados preliminares sobre las 5 017 vueltas de las 37 rutas muestran que LS supera a LPT cuando hay tráfico: con T1 deja 167 vueltas sin asignar frente a 331, y su brecha media respecto de la cota inferior es 13.7 % frente a 19.9 %. Con el tráfico pesimista T2, las vueltas no atendidas casi se triplican respecto de T0 (de 95 a 280). La ruta RTU-22 es infactible por capacidad aun sin tráfico ($\rho = 1.02$). En instancias pequeñas, CP-SAT confirma que elegir la hora de salida dentro de la ventana reduce la carga. También se detectó y corrigió un error de datos en RTU-28 y RTU-29.

Palabras clave: scheduling en máquinas paralelas, List Scheduling, LPT, tiempos de viaje dependientes del tiempo, ventanas temporales, ramificación y poda, CP-SAT, simulación Monte Carlo, transporte urbano.

Índice

Resumen.....	1
1. Introducción.....	4
1.1 Contexto y motivación.....	4
1.2 Observación recibida y respuesta.....	4
1.3 Problema.....	4
1.4 Objetivos.....	4
1.5 Alcance y delimitación.....	5
1.6 Organización del documento.....	5
2. Fundamentos y aprendizaje autónomo.....	6
2.1 Conceptos y algoritmos requeridos.....	6
2.2 Fuentes estudiadas y criterios de selección.....	7
2.3 Conocimientos nuevos que el grupo tuvo que adquirir.....	8
2.4 Plan de aprendizaje autónomo.....	8
3. Evaluación y selección del stack tecnológico.....	10
3.1 Alternativas y criterios.....	10
3.2 Procedimiento de aprendizaje e integración.....	11
3.3 Dificultades de adopción y soluciones.....	11

4. Formulación del problema.....	12
4.1 Elementos del modelo.....	12
4.2 Modelo del tráfico.....	12
4.3 Entradas.....	13
4.4 Salidas.....	13
4.5 Restricciones.....	14
4.6 Supuestos.....	14
4.7 Función objetivo y formulación matemática.....	14
4.8 Datos, instancias y control de calidad.....	15
5. Diseño algorítmico.....	16
5.1 Visión general.....	16
5.2 Estructuras de datos.....	16
5.3 Pseudocódigo.....	17
5.4 Invariantes y corrección.....	18
5.5 Análisis de complejidad.....	19
6. Diseño e implementación del sistema.....	20
6.1 Arquitectura y flujo de información.....	20
6.2 Módulos.....	20
6.3 Formatos de archivo.....	20
6.4 Dependencias y decisiones técnicas.....	21
7. Ejemplo manual paso a paso.....	22
7.1 Instancia.....	22
7.2 Cálculo a mano de la duración con tráfico.....	22
7.3 List Scheduling (orden por inicio de ventana).....	23
7.4 LPT adaptado.....	23
7.5 Referencias exactas y comparación.....	24
8. Estrategia de pruebas y validación.....	25
8.1 Validador independiente.....	25
8.2 Casos de prueba automatizados (pytest).....	25
8.3 ¿Cómo se verifica que una salida es correcta?.....	26
9. Metodología experimental.....	27
9.1 Preguntas de evaluación e hipótesis.....	27
9.2 Diseño de los experimentos.....	27
9.3 Entorno y procedimiento reproducible.....	27
10. Prototipo inicial y resultados preliminares.....	28
10.1 Repositorio.....	28
10.2 Funcionamiento del prototipo.....	28
10.3 E1 — Las 37 rutas bajo tráfico.....	29
10.4 E2 — Heurísticas frente a métodos exactos.....	30
10.5 E3 — Escalabilidad.....	31
10.6 E4 — Robustez ante la variabilidad del tráfico.....	31
10.7 Contraste con las hipótesis.....	32
11. Discusión, ajustes y dificultades.....	33
11.1 Interpretación y contraste con la teoría.....	33
11.2 Cuándo conviene cada alternativa.....	33
11.3 Ajustes realizados a partir de los avances anteriores.....	34
11.4 Dificultades pendientes.....	34
11.5 Amenazas a la validez.....	35
12. Conclusiones y recomendaciones.....	36
12.1 Conclusiones.....	36
12.2 Lecciones aprendidas.....	36
12.3 Trabajo para la Entrega 2.....	37
13. Referencias.....	38
Anexo A. Manual de instalación y uso.....	39
Anexo B. Matriz de contribuciones.....	40
Anexo C. Formato de bitácora individual.....	41
Anexo D. Correspondencia con los requisitos de la Entrega 1.....	42

Anexo E. Resultados por ruta.....	43
-----------------------------------	----

1. Introducción

1.1 Contexto y motivación

El transporte urbano de Cusco opera con empresas concesionarias que atienden una ruta fija con su propia flota. Cada día, la empresa debe decidir qué bus realiza cada vuelta y a qué hora sale. Para eso tiene que respetar la frecuencia comprometida, el descanso del conductor y una jornada limitada. Si las vueltas se reparten mal, unos buses acumulan muchas horas de conducción y otros quedan ociosos, y se pierden salidas en las horas de mayor demanda.

Desde el punto de vista algorítmico, la situación corresponde a un problema clásico de **programación de trabajos en máquinas paralelas idénticas** ($P||C_{\max}$), que es NP-difícil incluso con dos máquinas (Garey y Johnson, 1979). Sobre él se agregan restricciones propias del transporte: ventanas de inicio, periodos de indisponibilidad (almuerzo) y, a partir de esta entrega, **tiempos de viaje que dependen de la hora** por efecto del tráfico.

1.2 Observación recibida y respuesta

En la revisión de los avances, el docente preguntó **cómo se manejaría el tráfico en el transporte**. En el Avance 1 cada vuelta tenía una duración fija igual al tiempo de vuelta de la tabla 2020, de modo que una salida a las 07:00 duraba lo mismo que una a las 15:00. La respuesta adoptada en esta entrega tiene dos capas:

- **Capa A — tráfico determinístico.** La duración $p_i(s_j)$ depende de la hora de salida s_j . La ruta se recorre con una velocidad constante por franja horaria (punta mañana, mediodía y tarde; valles), siguiendo el modelo de Ichoua, Gendreau y Potvin (2003). El modelo cumple la propiedad FIFO: salir más tarde nunca hace llegar antes.
- **Capa B — variabilidad estocástica.** La duración realizada se multiplica por un ruido lognormal de media 1 ($\sigma = 0.10$ o 0.20). El plan se ejecuta por simulación Monte Carlo propagando los retrasos, y así se mide su robustez.

Con esto el tráfico deja de ser una suposición implícita y pasa a ser un factor experimental explícito, con tres escenarios (T0, T1, T2) que se comparan con las mismas métricas.

1.3 Problema

Dada una empresa/ruta con M buses idénticos, un conjunto de vueltas con su ventana de inicio, una jornada de 06:00 a 22:00, un almuerzo de 2 h por bus con ventana escalonada y un perfil de tráfico, se busca **asignar cada vuelta a un bus de su propia ruta y fijar su hora de salida**. El objetivo es atender la mayor cantidad de vueltas y, entre las soluciones que lo logran, minimizar la carga de conducción del bus más cargado ($L_{\max}$).

1.4 Objetivos

Objetivo general

Desarrollar y evaluar experimentalmente una solución computacional basada en algoritmos de programación en máquinas idénticas para asignar las vueltas de las rutas de transporte urbano de Cusco, incorporando ventanas de inicio, almuerzo y tiempos de viaje dependientes del tráfico. La solución debe demostrar aprendizaje autónomo, adopción justificada de tecnologías y pensamiento crítico al comparar alternativas.

Objetivos específicos

1. Formular el problema con entradas, salidas, restricciones, supuestos y función objetivo, incluyendo un modelo explícito del tráfico.
2. Investigar de forma autónoma los fundamentos de scheduling en máquinas paralelas, tiempos dependientes del tiempo, ramificación y poda y programación con restricciones.
3. Implementar al menos dos estrategias comparables (List Scheduling y LPT adaptados) y un mecanismo exacto de referencia (Ramificación y Poda y CP-SAT).
4. Validar la corrección con un verificador independiente, una instancia resuelta a mano y pruebas básicas, límite, adversas y de escala.
5. Diseñar experimentos reproducibles que comparen factibilidad, $L_{\max}$, equilibrio, tiempo de ejecución y robustez ante el tráfico.
6. Evaluar críticamente los datos, los supuestos, los resultados y las limitaciones.
7. Documentar el proceso de aprendizaje, las decisiones técnicas y la evolución del proyecto.

1.5 Alcance y delimitación

Incluido	Fuera de alcance
37 empresas/rutas de la tabla 2020; cada una es una instancia independiente.	Mezclar flotas o mover buses entre rutas.
Flota operativa; buses idénticos dentro de cada ruta.	Heterogeneidad de buses (capacidad, consumo, antigüedad).
Vuelta completa origen → destino → origen como trabajo indivisible.	Rediseño de rutas, paraderos o recorridos (VRP).
Jornada 06:00–22:00; almuerzo de 2 h con ventana escalonada en 3 turnos.	Reglas laborales detalladas (relevos, turnos partidos).
Tráfico por franjas (T0/T1/T2) y variabilidad estocástica simulada.	Simulación microscópica de tráfico (SUMO), ruteo en tiempo real o datos GPS.
Ventanas experimentales de la Opción C (± 15 / ± 30 min).	Horarios oficiales de las empresas (no disponibles).

Tabla 1. Alcance y delimitación del proyecto.

1.6 Organización del documento

La sección 2 presenta los fundamentos y el plan de aprendizaje autónomo. La sección 3 justifica el stack tecnológico. La sección 4 formula el problema reestructurado y la 5 detalla el diseño algorítmico, con pseudocódigo y complejidad. La sección 6 describe la arquitectura del sistema y la 7 resuelve paso a paso una instancia pequeña. Las secciones 8 y 9 presentan el plan de pruebas y la metodología experimental. La sección 10 muestra el prototipo y los resultados preliminares, la 11 discute ajustes, dificultades y amenazas a la validez, y la 12 concluye. Los anexos contienen el manual de uso, la matriz de contribuciones, el formato de bitácora y la correspondencia con los requisitos de la entrega.

2. Fundamentos y aprendizaje autónomo

2.1 Conceptos y algoritmos requeridos

Programación en máquinas paralelas idénticas

En la notación de tres campos de Graham, $P||C_{\max}$ consiste en repartir n trabajos con tiempos p_i entre m máquinas idénticas para minimizar el instante en que termina la última (makespan). Sin restricciones temporales, el makespan coincide con la **carga máxima** de una máquina. El problema es NP-difícil (Garey y Johnson, 1979), por eso se usan heurísticas con garantía:

- **List Scheduling (LS)** recorre los trabajos en un orden dado y coloca cada uno en la máquina menos cargada. Garantiza $C_{\max} \leq (2 - 1/m) \cdot C^*$ (Graham, 1966).
- **LPT (Longest Processing Time first)** ordena antes los trabajos de mayor duración. Garantiza $C_{\max} \leq (4/3 - 1/(3m)) \cdot C^*$ (Graham, 1969).

Precaución. Las garantías de Graham **no se trasladan** a nuestro problema. Con ventanas de inicio y almuerzo, una mala decisión temprana puede dejar trabajos sin asignar, y con tráfico la duración depende de la hora. Por eso es necesario comparar experimentalmente y usar un método exacto como referencia en instancias pequeñas.

Ventanas e indisponibilidad de máquinas

Cuando cada trabajo solo puede iniciar en un intervalo $[r_i, d_i]$, el problema se acerca a la **programación de intervalos** (Kolen et al., 2007). En ese caso decidir si todos los trabajos caben ya es difícil. El almuerzo convierte a cada bus en una máquina con un **periodo de indisponibilidad** (Lee, 1996). Al darle una ventana de inicio, el almuerzo pasa a ser un trabajo fijo a su bus que también hay que programar.

Tiempos de viaje dependientes del tiempo y propiedad FIFO

En la programación dependiente del tiempo (Gawiejnowicz, 2008), la duración de un trabajo es una función de su inicio. Para el transporte, Ichoua, Gendreau y Potvin (2003) proponen dividir el día en franjas con velocidad constante y calcular el tiempo de viaje integrando la velocidad tramo por tramo. A diferencia de multiplicar la duración por el factor de la franja de salida, este modelo cumple la propiedad **FIFO**: si $s_1 \leq s_2$, entonces $s_1 + p(s_1) \leq s_2 + p(s_2)$. En el diseño esto tiene una consecuencia práctica: dentro de un hueco libre, la salida más temprana es también la que llega antes (sección 5.4).

Métodos exactos de referencia

- **Ramificación y poda** (Land y Doig, 1960): explora el árbol de asignaciones y descarta las ramas cuya cota inferior no puede mejorar la mejor solución conocida.
- **Programación con restricciones (CP-SAT)**: el solver de Google OR-Tools combina propagación de restricciones, SAT y búsqueda; representa las vueltas como intervalos opcionales con restricciones de no solapamiento (*NoOverlap*) por bus.

Simulación Monte Carlo

Para medir la robustez se repite la ejecución del plan con duraciones aleatorias y semillas fijas, y se resumen las medias y el percentil 95 de las métricas (Law, 2015).

2.2 Fuentes estudiadas y criterios de selección

Se priorizaron artículos con revisión por pares, libros de referencia y documentación oficial. Cada fuente se evaluó por **pertinencia** (¿modela algo de nuestro problema?), **confiabilidad** (revisión por pares o autoría reconocida), **vigencia** (si el resultado sigue siendo la referencia del área) y **verificabilidad** (si se puede reproducir con nuestros datos).

Fuente	Tipo	Aporte al proyecto	Uso
Graham (1966, 1969)	Artículos (BSTJ, SIAM J. Appl. Math.)	Cotas de LS y LPT	Fundamento y prueba del peor caso de LPT
Pinedo (2016)	Libro	Notación, reglas de despacho, $P C_{\max}$	Formulación y métricas
Garey y Johnson (1979)	Libro	NP-dificultad	Justifica usar heurísticas
Ichoua, Gendreau y Potvin (2003)	Artículo (EJOR)	Velocidad por franjas y FIFO	Modelo de tráfico (capa A)
Gawiejnowicz (2008)	Libro	Scheduling dependiente del tiempo	Marco teórico del tráfico
Lee (1996)	Artículo (J. Global Optim.)	Máquinas con indisponibilidad	Modelo del almuerzo
Kolen et al. (2007)	Artículo (Naval Res. Logist.)	Interval scheduling	Factibilidad con ventanas
Ibarra-Rojas et al. (2015)	Revisión (Transp. Res. B)	Planificación de buses; robustez	Contexto y holguras
Land y Doig (1960)	Artículo (Econometrica)	Ramificación y poda	Diseño del B&B
Documentación de OR-Tools	Documentación oficial	CP-SAT, intervalos, AddElement	Modelo exacto
Law (2015)	Libro	Diseño de simulaciones, réplicas	Capa B y experimento E4
Tabla de flota operativa 2020	Dato referencial (Scribd)	Parámetros de las 37 rutas	Instancias (con control de calidad)

Tabla 2. Fuentes principales, su tipo y su uso en el proyecto.

Evaluación crítica de la fuente de datos. La tabla de flota 2020 es un documento secundario publicado en un repositorio abierto, no un dataset oficial. Por eso se usa solo como **referencia de parámetros**, se verificó su consistencia interna (sección 4.8) y todo dato construido para experimentar se marca como tal.

2.3 Conocimientos nuevos que el grupo tuvo que adquirir

Conocimiento	Por qué fue necesario	Dónde se aplica
Tiempos de viaje dependientes del tiempo (FIFO)	Responder a la observación sobre el tráfico	traffic.py, hoja <i>Calculadora vuelta</i>
Programación con restricciones (CP-SAT)	Obtener óptimos de referencia con inicio flexible	algorithms/cpsat.py
Máquinas con indisponibilidad flexible	Almuerzo fijo generaba infactibilidad artificial	BusTimeline
Diseño de cotas y simetrías en B&B	Hacer manejable la búsqueda exacta	algorithms/bnb.py
Simulación Monte Carlo y percentiles	Medir robustez ante tráfico variable	simulation.py, E4
Pruebas automatizadas con pytest	Verificar restricciones y casos límite	tests/
Control de calidad de datos con pandas/openpyxl	Detectar inconsistencias entre hojas	builder.py

Tabla 3. Conocimientos nuevos y su aplicación.

2.4 Plan de aprendizaje autónomo

El plan asigna cada necesidad de aprendizaje a un responsable, con actividades independientes, fuentes y la evidencia verificable esperada. Cada integrante registra el avance real en su bitácora individual (docs/bitacora/, formato en el Anexo C). Así se cubre el indicador AG-C06.01, que se evalúa de forma individual.

#	Necesidad (brecha)	Responsable	Actividad independiente	Evidencia
N1	Tiempo de viaje por franjas y FIFO	Aguilar	Leer Ichoua et al.; 3 casos a mano; implementar y probar FIFO	traffic.py, test_traffic.py
N2	CP-SAT (intervalos opcionales, NoOverlap)	Aguilar	Tutorial oficial; modelo de juguete; modelo completo	cpsat.py, E2
N3	$P C_{\max}$ y cotas de Graham	Choque	Graham y Pinedo; reproducir peor caso de LPT	greedy.py, prueba de Graham
N4	Estructuras para huecos de tiempo	Choque	Comparar lista ordenada + bisect con árbol de intervalos	BusTimeline, E3
N5	Cotas y simetrías en B&B	Huacani	Land y Doig; implementar cota $\Sigma p_{\min}/m$ y poda por simetría	bnb.py, nodos/podas
N6	Pruebas límite y adversas	Huacani	Guía de pytest; catálogo de casos	tests/ (35 pruebas)
N7	Calidad de datos (pandas/openpyxl)	Moreano	Cruce de hojas del Excel; detección de RTU-28/29	builder.py, Excel
N8	Monte Carlo y robustez	Moreano	Law, caps. 4 y 9; réplicas con semilla	simulation.py, E4
N9	Visualización reproducible	Moreano	Gantt y gráficos con paleta accesible	viz.py, figuras

Tabla 4. Plan de aprendizaje autónomo (detalle en docs/plan_aprendizaje.md).

Estrategia de evidencia. Cada actividad deja tres rastros verificables: una fila fechada en la bitácora individual, un commit identificable en el repositorio y una prueba o figura que demuestra la

aplicación. En las sesiones de seguimiento cada integrante muestra su parte y explica qué aprendió y qué dificultad resolvió.

3. Evaluación y selección del stack tecnológico

3.1 Alternativas y criterios

Las alternativas se compararon con cinco criterios, cada uno puntuado de 1 a 5: **pertinencia** para el problema, **curva de aprendizaje** para el grupo, **reproducibilidad** (instalación sin licencias ni configuraciones locales), **rendimiento** en instancias de hasta unas 200 vueltas por ruta e **integración** con el resto del stack.

Necesidad	Alternativas	Seleccionada	Justificación
Lenguaje	Python 3 · C++ · Java	Python 3.13	Stack de referencia del curso; las heurísticas tardan milisegundos por ruta (E3), por lo que la velocidad de C++ no compensa su costo de desarrollo.
Solver exacto	B&B propio · PuLP/CBC (MILP) · Gurobi · OR-Tools CP-SAT	CP-SAT + B&B propio	CP-SAT modela de forma nativa intervalos opcionales, no solapamiento y duraciones que dependen del inicio (AddElement); es libre y se instala con pip. Gurobi requiere licencia. El B&B propio se mantiene como evidencia de dominio del algoritmo.
Modelo de tráfico	Factor por franja de salida · velocidad por franjas (IGP) · SUMO	Velocidad por franjas (IGP)	Garantiza FIFO con muy poco cálculo. SUMO exige red vial, demanda y calibración que no existen para Cusco, y queda fuera del alcance.
Datos	pandas · openpyxl · csv	pandas + openpyxl	Lectura y escritura del Excel con fórmulas; análisis tabular de resultados.
Visualización	matplotlib · plotly	matplotlib	Figuras estáticas para el informe; sin dependencias de navegador.
Pruebas	unittest · pytest	pytest	Pruebas parametrizadas e importación de dependencias opcionales (importorskip).
Versionado	Git + GitHub	Git	Historial de contribuciones por integrante.
Documentación	LaTeX · Word · README	Word + README	Word por requerimiento de esta entrega; README con instalación y comandos.

Tabla 5. Matriz de comparación y selección tecnológica.

Criterio (1–5)	B&B propio	PuLP/CBC	Gurobi	CP-SAT
Pertinencia (intervalos, AddElement)	3	3	4	5
Curva de aprendizaje	4	4	3	3
Reproducibilidad (licencia, pip)	5	5	2	5
Rendimiento en instancias pequeñas	3	3	5	4
Integración con Python	5	4	4	5
Total	20	19	18	22

Tabla 6. Puntuación de las alternativas para el método exacto.

3.2 Procedimiento de aprendizaje e integración

1. Prototipo de juguete en CP-SAT con 3 vueltas y 2 buses para entender intervalos opcionales y la restricción de no solapamiento.
2. Incorporación de AddElement para que la duración dependa del minuto de salida: se tabula $p_i(t)$ para cada t de la ventana.
3. Verificación cruzada con el B&B propio en la instancia manual y en 90 instancias aleatorias (E2).
4. Integración en la CLI (`--alg cpsat`) y en las pruebas, con `pytest.importorskip` para que el resto funcione aunque OR-Tools no esté instalado.

3.3 Dificultades de adopción y soluciones

Dificultad	Solución aplicada
CP-SAT solo trabaja con enteros, pero las duraciones con tráfico son reales.	Discretización a 1 minuto, redondeando la duración hacia arriba (conservador). La solución se valida luego con la duración exacta.
Con varios hilos CP-SAT devuelve soluciones distintas de igual valor.	1 hilo y semilla fija para el ejemplo manual; varios hilos solo donde basta el valor óptimo.
Simetría: muchos buses idénticos generan soluciones equivalentes.	Orden de cargas entre buses con el mismo turno de almuerzo (CP-SAT) y poda de buses vacíos equivalentes (B&B).
Lectura del Excel: hojas inconsistentes en RTU-28 y RTU-29.	Control de calidad automático en <code>builder.py</code> y regeneración con la regla de la Opción C (sección 4.8).

Tabla 7. Dificultades de adopción tecnológica.

4. Formulación del problema

4.1 Elementos del modelo

Elemento	Definición
Instancia	Una empresa/ruta r . Las 37 rutas se resuelven por separado; los buses no cambian de ruta.
Máquina i	Un bus de la flota operativa de r . Buses idénticos en velocidad; difieren solo en el turno de almuerzo.
Trabajo j	Una vuelta completa origen $\rightarrow$ destino $\rightarrow$ origen, con duración base $\bar{p}_j$ (tabla 2020) y ventana de inicio $[r_j, d_j]$.
Duración con tráfico	$p_i(s)$ = tiempo real para recorrer $\bar{p}_j$ minutos base saliendo en s (capa A).
Almuerzo	Bloque de 120 min por bus que debe iniciar en $[a_i, b_i]$: turnos 10:30–11:30, 12:00–13:00 y 13:30–14:30 (bus $k \rightarrow$ turno $k \bmod 3$).
Jornada	[06:00, 22:00]: toda vuelta inicia y termina dentro de ella.
Carga L_i	Suma de las duraciones (con tráfico) de las vueltas del bus i .

Tabla 8. Elementos del modelo.

4.2 Modelo del tráfico

Sea el día dividido en franjas $[\tau_k, \tau_{k+1})$ con factor de congestión $f_k > 0$. Dentro de la franja k , un minuto real avanza $1/f_k$ minutos del recorrido base. La duración $p_i(s)$ es el menor Δ tal que:

$$\int_s^{s+\Delta} dt / f(t) = \bar{p}_j$$

Como f es constante por tramos, la integral se resuelve franja por franja: se consume la capacidad $(\tau_{k+1} - t)/f_k$ de cada franja hasta completar $\bar{p}_j$. Se definieron tres escenarios:

Franja	Condición	T2 (pesimista)	T1 (normalizado)	T0
06:00–06:30	Valle temprano	0.90	0.798	1
06:30–09:00	Punta mañana	1.30	1.152	1
09:00–12:00	Valle	1.00	0.886	1
12:00–14:30	Punta mediodía (salida escolar)	1.20	1.064	1
14:30–17:00	Valle	1.00	0.886	1
17:00–20:00	Punta tarde	1.35	1.197	1
20:00–22:00	Valle nocturno	0.90	0.798	1
Media ponderada		1.128	1.000	1

Tabla 9. Perfiles de congestión. $T1 = T2 / 1.128$, de modo que la media del día sea 1.

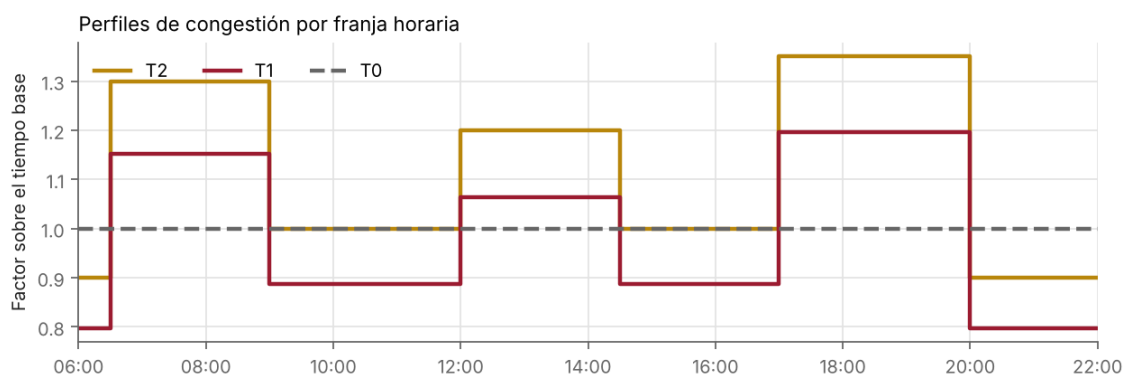


Figura 1. Perfiles de congestión T0, T1 y T2 a lo largo de la jornada.

Justificación de T1. El tiempo de vuelta de la tabla 2020 se calculó con una velocidad media (15–20 km/h). Normalizar el perfil a media 1 hace que la duración promedio del día coincida con ese dato, y el tráfico solo **redistribuye** el tiempo entre puntas y valles. T2 no se normaliza y representa un día de congestión generalizada (+12.8 % en promedio).

Capa B. En la simulación, la duración realizada es $p_i(s) \cdot \varepsilon$ con $\varepsilon \sim \text{LogNormal}(-\sigma^2/2, \sigma)$, cuya media es 1 y $\sigma \in \{0.10, 0.20\}$. Un bus no sale antes de su hora programada y, si llega tarde, sale tarde a la siguiente vuelta: inicio real = max(inicio planificado, fin real anterior). El almuerzo también se desplaza.

Aclaración. Los factores son **supuestos experimentales**: no existe un índice público de congestión para Cusco. La hoja *Perfil trafico* del Excel y los archivos data/traffic/*.json permiten modificarlos sin tocar el código. Calibrarlos con observaciones propias queda como trabajo de la Entrega 2.

4.3 Entradas

Entrada	Descripción	Origen
Ruta, empresa, flota M	Identificador, nombre y número de buses	Tabla 2020
Duración base $\bar{p}_i$	Tiempo de vuelta de la ruta (h → min)	Tabla 2020
Vueltas planificadas n	round($M \times$ viajes por unidad vehicular)	Construcción del proyecto
Ventana $[r_i, d_i]$	Opción C: ± 15 min en 06–09 y 16–19; ± 30 min en el resto	Experimental
Jornada	06:00–22:00	Supuesto
Almuerzo	120 min; 3 turnos con ventana de ± 30 min	Supuesto
Perfil de tráfico	Franjas y factores (T0/T1/T2 o archivo JSON)	Experimental
Parámetros de ejecución	Algoritmo, política de salida, holgura, σ , réplicas, semilla	Usuario (CLI)

Tabla 10. Entradas del modelo.

4.4 Salidas

- Bus asignado, hora de salida y hora de llegada de cada vuelta, o la marca NO_ASIGNADO.
- Hora de almuerzo de cada bus.

- Carga por bus, $L_{\max}$, $L_{\min}$, desbalance, utilización, sobre costo por tráfico y brecha respecto de la cota inferior.
- Resultado del validador (lista de violaciones) y tiempo de ejecución.
- En simulación: % de vueltas fuera de ventana, retraso medio y máximo, % de buses con horas extra (media y p95).
- Archivos CSV de asignaciones y diagramas de Gantt.

4.5 Restricciones

1. **Ruta:** una vuelta solo puede asignarse a un bus de su propia ruta.
2. **Asignación única:** cada vuelta se asigna a lo sumo a un bus (o queda no asignada).
3. **Ventana:** $r_j \leq s_j \leq d_j$.
4. **Duración con tráfico:** $C_j = s_j + p_j(s_j)$.
5. **Jornada:** $06:00 \leq s_j$ y $C_j \leq 22:00$.
6. **No solapamiento:** dos vueltas del mismo bus no se superponen.
7. **Almuerzo:** cada bus tiene exactamente un bloque de 120 min que inicia en su ventana y no se superpone con sus vueltas.

4.6 Supuestos

1. Los buses de una ruta son idénticos en velocidad y capacidad.
2. La vuelta es indivisible y su duración base es la de la tabla 2020.
3. El tráfico afecta por igual a toda la ruta en una franja dada (sin variación espacial).
4. Las ventanas y los turnos de almuerzo son políticas experimentales reproducibles.
5. En la capa B, las perturbaciones de distintas vueltas son independientes.
6. No hay tiempos muertos de terminal entre vueltas (se pueden modelar como holgura).

4.7 Función objetivo y formulación matemática

Corrección respecto del Avance 1. En el Avance 1 se minimizaba $C_{\max}$ (la hora de fin de la última vuelta), pero en el ejemplo manual se reportaba en realidad la carga máxima (5.2 h, cuando las vueltas terminaban a las 11:12). Con ventanas repartidas en toda la jornada, la última vuelta siempre termina cerca de las 22:00 sin importar la asignación, así que ese objetivo no discrimina entre soluciones. Por eso se adopta $L_{\max}$, que es el análogo exacto del makespan de $P||C_{\max}$.

Además, algunas rutas no permiten atender todas las vueltas (sección 4.8), de modo que el objetivo es **lexicográfico**:

$$\min (U, L_{\max}), \quad U = \sum_j u_j \text{ (vueltas no asignadas)}, \quad L_{\max} = \max_i \sum_j x_{ij} \cdot p_j(s_j)$$

Componente	Expresión
Variables	$x_{ij} \in \{0,1\}; u_j \in \{0,1\}; s_j \in [r_j, d_j]; \ell_i \in [a_i, b_i]$ (inicio del almuerzo)
Asignación	$\sum_i x_{ij} + u_j = 1 \quad \forall j$
Duración	$p_j = p_j(s_j)$ (tabla discretizada por minuto en CP-SAT)
Jornada	$s_j + p_j(s_j) \leq 22:00$
No solapamiento	$\text{NoOverlap}(\{[s_j, s_j+p_j] : x_{ij}=1\} \cup \{[\ell_i, \ell_i+120]\}) \quad \forall i$
Carga	$L_i = \sum_j x_{ij} \cdot p_j; \quad L_{\max} \geq L_i$
Objetivo (pesos)	$\min \text{BIG} \cdot U + L_{\max}, \text{ con } \text{BIG} > m \cdot 960 \text{ para que } U \text{ domine}$

Tabla 11. Formulación matemática (implementada en algorithms/cpsat.py).

Cota inferior. Sea $p_j^{\min}$ la menor duración posible de j dentro de su ventana. Entonces $L_{\max} \geq LB = \max(\max_j p_j^{\min}, \sum_j p_j^{\min} / m)$. Esta cota se usa para medir la brecha de las heurísticas en las 37 rutas y para podar en el B&B.

Métricas complementarias. Desbalance $L_{\max} - L_{\min}$; utilización = $\sum L_i / (m \cdot 840 \text{ min})$; sobrecosto por tráfico = $(\sum p_j(s_j) - \sum \bar{p}_j) / \sum \bar{p}_j$; tiempo de ejecución; y las métricas de robustez de la capa B.

4.8 Datos, instancias y control de calidad

Las instancias se construyen desde el Excel del proyecto. *Parametros del modelo* aporta la flota, la duración base y las vueltas planificadas, y *Trabajos y ventanas* aporta cada vuelta con su ventana. El constructor verifica que ambas hojas sean consistentes.

Hallazgo de calidad de datos. En **RTU-28** y **RTU-29**, la hoja *Tabla proyecto* tiene las columnas desplazadas: demanda/h = 2.34 y viajes por unidad = 98.6 (y 2.58 y 94.52), valores imposibles frente a 2.9–6.7 en el resto de rutas. *Parametros del modelo* ya tenía el valor corregido (145 y 139 vueltas), pero *Trabajos y ventanas* se había generado con el valor erróneo: **3 057 y 2 836 vueltas**, el 55 % de todas las vueltas del proyecto. El constructor detecta la discrepancia y regenera las vueltas con la misma regla de la Opción C; la regla se verificó contra RTI-01 con diferencias ≤ 1 min. Sin esta corrección, más de la mitad de las vueltas aparecían como infactibles.

Tras la corrección, el proyecto tiene **5 017 vueltas** en 37 rutas (103 a 166 por ruta, con 16 a 44 buses). Para anticipar la dificultad se define la **saturación** de la flota:

$$\rho = \sum_j \bar{p}_j / (M \cdot 14 \text{ h})$$

$\rho > 1$ implica que la ruta es infactible aun sin tráfico ni ventanas. **RTU-22** (19 buses, 127 vueltas de 2.13 h) tiene $\rho = 1.02$: necesita 270.5 h y la flota ofrece 266 h. RTI-02 ($\rho = 0.87$), RTU-05 (0.82) y RTU-01 (0.79) son las siguientes más ajustadas. La hoja *Saturacion flota* del Excel calcula ρ con fórmulas.

5. Diseño algorítmico

5.1 Visión general

Las heurísticas siguen un esquema común de **lista con inserción en huecos**. Se recorre una lista de vueltas en un orden que define la estrategia. Para cada vuelta se busca en cada bus el hueco libre más temprano donde quepa, respetando la ventana, la jornada y la posibilidad de seguir ubicando el almuerzo. Luego se elige un bus según una regla de selección. Las estrategias difieren en el **orden** de la lista:

Estrategia	Orden de la lista	Regla de selección de bus	Rol
LS (List Scheduling)	Inicio mínimo de ventana r_i (orden de servicio)	Menor carga acumulada; empate → salida más temprana → índice	Heurística 1
LPT adaptado	Duración con tráfico en la salida nominal, de mayor a menor	Igual que LS	Heurística 2
B&B	Cronológico (r_i), ramificando sobre los buses	Exploración exhaustiva con poda	Exacto (instancias pequeñas)
CP-SAT	— (modelo declarativo)	Solver	Exacto de referencia
LS-BF (variante)	Como LS	Menor tiempo muerto antes de la vuelta (<i>best fit</i>)	Explorada y descartada (sección 11)

Tabla 12. Estrategias implementadas.

Además, LS y LPT aceptan dos parámetros que responden directamente al tráfico. La **política de salida** puede ser *earliest* (lo antes posible) o *min_duration* (dentro del hueco, el minuto que minimiza la duración con tráfico). La **holgura** (h) planifica con duraciones infladas en $(1 + h)$ para absorber la variabilidad.

5.2 Estructuras de datos

Estructura	Implementación	Operaciones y costo
Job	dataclass inmutable: id, ruta, $\bar{p}$, r , d , nominal	acceso $O(1)$
Bus	dataclass: id, ruta, ventana de almuerzo $[a, b]$, duración	acceso $O(1)$
TrafficProfile	lista ordenada de franjas (inicio, fin, factor)	$\text{travel_time } O(F)$, $F = 7$ franjas
BusTimeline	tres listas paralelas ordenadas (inicios, fines, etiquetas) + carga	huecos $O(k)$; inserción <i>bisect</i> $O(\log k)$ + desplazamiento $O(k)$
Schedule	diccionario vuelta → Assignment, lista de no asignadas, almuerzos	consulta $O(1)$, agrupación por bus $O(n \log n)$
Instance	lista de vueltas + lista de buses + índice por id	serialización JSON $O(n + m)$

Tabla 13. Estructuras de datos (k = vueltas ya asignadas al bus).

Se eligió una lista ordenada con *bisect* en lugar de un árbol de intervalos porque cada bus tiene a lo sumo unas 6 vueltas al día. Con k tan pequeño, el costo $O(k)$ de desplazar elementos es

despreciable y el código es más fácil de verificar. La alternativa se documenta como mejora si el número de vueltas por bus creciera.

5.3 Pseudocódigo

Duración con tráfico (capa A)

```
función TIEMPO_VIAJE(s,  $\bar{p}$ , franjas):
    restante  $\leftarrow \bar{p}$  ; t  $\leftarrow$  s
    para cada franja ( $\tau_{ini}$ ,  $\tau_{fin}$ , f) con  $\tau_{fin} > s$ , en orden:
        si t <  $\tau_{ini}$ : (hueco sin franja) usar f_fuera hasta  $\tau_{ini}$ 
        capacidad  $\leftarrow (\tau_{fin} - t) / f$   $\triangleright$  minutos base que caben
        si capacidad  $\geq$  restante: devolver (t + restante·f) - s
        restante  $\leftarrow$  restante - capacidad ; t  $\leftarrow \tau_{fin}$ 
    devolver (t + restante·f_fuera) - s
```

Búsqueda del hueco factible en un bus

```
función BUSCAR_HUECO(bus, j, dur, política):
    para cada hueco [g0, g1] libre del bus, en orden cronológico:
        s0  $\leftarrow$  max(r_j, g0) ; si s0 > d_j: devolver NINGUNO
        candidatos  $\leftarrow$  [s0, max(s0, max(g0, a_bus) + 120)]  $\triangleright$  salir ya, o almorzar
    antes
        para s en candidatos:
            si s > d_j o s + dur(s) > g1: romper  $\triangleright$  FIFO: más tarde tampoco cabe
            si ALMUERZO_CABE(bus u [s, s+dur(s)]):
                si política = earliest: devolver (s, s+dur(s))
                si no: devolver el t  $\in$  [s, min(d_j, g1)] de menor dur(t) que quepa
    devolver NINGUNO
```

List Scheduling y LPT adaptados

```
función GREEDY(instancia, orden, perfil, política, holgura):
    para cada bus i: L_i  $\leftarrow$  0 ; línea_i  $\leftarrow$  vacía
    para cada vuelta j en orden:
        dur  $\leftarrow$  t  $\mapsto$  TIEMPO_VIAJE(t,  $\bar{p}_j$ , perfil)·(1 + holgura)
        mejor  $\leftarrow$  NINGUNO
        para cada bus i de la ruta:
            h  $\leftarrow$  BUSCAR_HUECO(i, j, dur, política)
            si h  $\neq$  NINGUNO y (L_i, h.inicio, i) < clave(mejor): mejor  $\leftarrow$  (i, h)
        si mejor = NINGUNO: marcar j como NO_ASIGNADA
        si no: insertar j en línea_i ; L_i  $\leftarrow$  L_i + duración
    para cada bus: ubicar almuerzo en el inicio factible más temprano

LS : orden  $\leftarrow$  vueltas ordenadas por (r_j, nominal_j)
LPT: orden  $\leftarrow$  vueltas ordenadas por -TIEMPO_VIAJE(nominal_j,  $\bar{p}_j$ )
```

Ramificación y poda

```

función BNB(instancia):
    incumbente ← mejor(LS, LPT) ; (U*, L*) ← valor(incumbente)
    pmin_j ← min_{t∈[r_j, d_j]} TIEMPO_VIAJE(t, p̄_j) ; suf_k ← Σ_{q≥k} pmin_q
    RAMIFICAR(k = 0, U = 0, total = 0)

función RAMIFICAR(k, U, total):
    LB ← max(max_i L_i, (total + suf_k) / m)
    si (U, LB) ≥ (U*, L*): podar
    si k = n: actualizar incumbente ; volver
    para cada bus i (omitiendo buses vacíos con el mismo turno de almuerzo):
        h ← BUSCAR_HUECO(i, j_k, earliest)
        si h: insertar ; RAMIFICAR(k+1, U, total + dur) ; deshacer
    si U + 1 ≤ U*: RAMIFICAR(k+1, U+1, total)      ▷ dejar j_k sin asignar

```

El B&B es **exacto dentro del espacio de programas cronológicos de salida más temprana**, que es el mismo espacio que explora LS. CP-SAT es exacto sobre el modelo completo, porque puede retrasar una salida dentro de la ventana. La diferencia entre ambos mide cuánto vale decidir la hora de salida, y en la instancia manual es de 4.6 min (sección 7).

Simulación Monte Carlo (capa B)

```

función SIMULAR(plan, perfil, σ, R, semilla):
    para r = 1..R:
        para cada bus: t ← 06:00
            para cada evento (vuelta o almuerzo) en orden planificado:
                inicio ← max(inicio_planificado, t)
                si almuerzo: t ← inicio + 120 ; continuar
                registrar retraso = inicio - planificado ; fuera = inicio > d_j
                ε ← exp(N(-σ²/2, σ)) ; t ← inicio + TIEMPO_VIAJE(inicio, p̄_j)·ε
                horas_extra ← (t > 22:00)
    devolver media y p95 de cada métrica

```

5.4 Invariantes y corrección

- **I1 — Factibilidad parcial.** Después de cada inserción, toda línea de tiempo cumple ventana, jornada y no solapamiento, y todavía existe un hueco donde cabe el almuerzo. `BUSCAR_HUECO` solo acepta candidatos que pasan `ALMUERZO_CABE`, de modo que el invariante se mantiene por inducción sobre las inserciones.
- **I2 — Optimalidad local de la salida.** Por la propiedad FIFO, $s \mapsto s + p(s)$ es no decreciente. Dentro de un hueco, el candidato más temprano termina antes que cualquier otro, y si no cabe ninguno posterior cabe. Esto justifica el romper del pseudocódigo.
- **I3 — Conservación.** Cada vuelta termina asignada a un solo bus o en la lista de no asignadas. El validador lo comprueba de forma independiente.
- **I4 — Cota válida.** $LB \leq L_{\max}$ en cualquier solución que asigne todas las vueltas, porque cada vuelta dura al menos $p^{\min}_j$ y la carga total se reparte entre m buses. El B&B solo poda cuando ni siquiera la cota puede mejorar al incumbente.

5.5 Análisis de complejidad

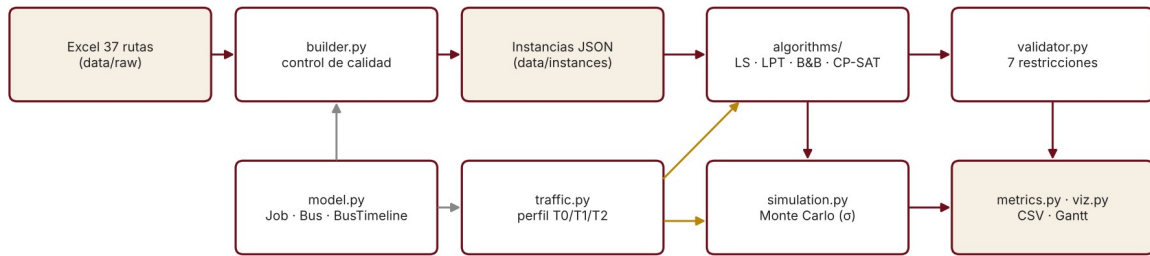
Componente	Tiempo	Espacio	Comentario
TIEMPO_VIAJE	$O(F)$	$O(1)$	$F = 7$ franjas $\rightarrow$ constante
BUSCAR_HUECO (earliest)	$O(k)$	$O(k)$	$k \approx n/m$ vueltas por bus; ALMUERZO_CABE es $O(k)$
BUSCAR_HUECO (min_duration)	$O((w/\Delta t) \cdot k)$	$O(k)$	$w \leq 60$ min de ventana, $\Delta t = 1$ min
LS / LPT	$O(n \log n + n \cdot m \cdot k) = O(n^2)$	$O(n + m)$	Con $m \propto n$; en una ruta real ($n \leq 166$) tarda ≈ 15 ms
B&B	$O(m^n)$ en el peor caso	$O(n + m)$	Poda + simetría; usado con $n \leq 12$
CP-SAT	Exponencial en el peor caso	$O(n \cdot m + \sum_i w_i)$	$n \cdot m$ booleanos; tabla de duraciones por minuto
Validador	$O(n \log n)$	$O(n)$	Ordena por bus y revisa vecinos
Simulación	$O(R \cdot n)$	$O(n)$	R réplicas

Tabla 14. Complejidad de los componentes.

El costo cuadrático de LS y LPT se debe a que cada vuelta prueba todos los buses. El experimento E3 lo confirma: al duplicar n (con $m = n/3$) el tiempo crece unas 4 veces (de 5.4 s a 21.6 s entre $n = 2\,000$ y $n = 4\,000$). Como cada ruta real tiene menos de 170 vueltas, el costo es irrelevante en la práctica: las 37 rutas se resuelven en menos de 0.7 s en total.

6. Diseño e implementación del sistema

6.1 Arquitectura y flujo de información



Flechas guinda: datos de la instancia · doradas: modelo de tráfico · grises: estructuras compartidas

Figura 2. Módulos del sistema y flujo de información.

El flujo es lineal y cada paso deja un archivo intermedio, lo que facilita la reproducibilidad. El Excel se transforma en instancias JSON (con control de calidad); cada instancia se resuelve con el algoritmo elegido y el perfil de tráfico; el validador independiente revisa el programa; y las métricas, los CSV y los diagramas de Gantt resumen el resultado. La simulación toma el plan y lo ejecuta bajo tráfico estocástico.

6.2 Módulos

Módulo	Responsabilidad
traffic.py	Perfiles T0/T1/T2, travel_time por integración por franjas, normalización, carga y guardado JSON.
model.py	Job, Bus, Instance, BusTimeline (huecos, almuerzo flexible, inserción) y Schedule.
builder.py	Excel → instancias, control de calidad y regeneración (Opción C), instancia manual y generador aleatorio con semilla.
algorithms/greedy.py	LS, LPT y la variante best-fit; políticas de salida y holgura.
algorithms/bnb.py	Ramificación y poda con cota, simetría e incumbente heurístico.
algorithms/cpsat.py	Modelo exacto en OR-Tools CP-SAT.
validator.py	Siete verificaciones independientes: cobertura, ruta, ventana, jornada, duración con tráfico, almuerzo y no solapamiento.
metrics.py	$L_{\max}$, desbalance, utilización, sobrecosto, cota inferior, brecha y saturación p.
simulation.py	Monte Carlo con propagación de retrasos.
viz.py	Gantt por bus con franjas de tráfico de fondo.
cli.py	Comandos build, run, demo, simulate y profiles.

Tabla 15. Módulos del paquete sched.

6.3 Formatos de archivo

Instancia (JSON). Una por ruta en data/instances/. Horas en formato HH:MM y duraciones en minutos:

```
{
  "route": "RTI-01",
  "company": "E.T. SAYLLA S.A.",
  "day_start": "06:00",
  "day_end": "22:00",
  "buses": [
    {
      "id": "RTI-01-B1",
      "lunch_r": "10:30",
      "lunch_d": "11:30",
      "lunch_min": 120,
      ...
    }
  ],
  "jobs": [
    {
      "id": "RTI-01-J001",
      "base_min": 148.8,
      "r": "06:00",
      "d": "06:15",
      "nominal": "06:00",
      ...
    }
  ],
  "meta": {
    "duracion_base_h": 2.48,
    "viajes_planificados": 103
  }
}
```

Perfil de tráfico (JSON). {"name": "T2", "outside_factor": 0.9, "bands": [{"start": "06:30", "end": "09:00", "factor": 1.3, "label": "Punta mañana"}, ...]}. Se puede usar un perfil propio con --traffic archivo.json.

Salida (CSV). Una fila por vuelta: ruta, trabajo, bus, ventana_min, ventana_max, inicio, fin, duracion_min, duracion_base_min. Las vueltas no asignadas llevan bus = NO_ASIGNADO.

Excel actualizado. Conserva las hojas originales (con RTU-28/29 corregidas y marcadas) y agrega *Perfil trafico*, *Calculadora vuelta*, *Saturacion flota*, *Instancia manual*, *Resultados por ruta* y *Resumen experimentos*, con 536 fórmulas recalculadas y sin errores.

6.4 Dependencias y decisiones técnicas

- Dependencias: Python ≥ 3.10 , openpyxl, pandas, matplotlib, ortools (opcional para CP-SAT) y pytest.
- El núcleo (modelo, heurísticas, B&B, validador y simulación) usa solo la biblioteca estándar. pandas y matplotlib se usan para experimentos y figuras.
- Todas las rutas son relativas al repositorio; no hay rutas absolutas ni configuraciones locales (instrucción adicional 13 de la guía).
- Los parámetros experimentales (perfiles, turnos de almuerzo, semillas) son explícitos y modificables sin editar los algoritmos.

7. Ejemplo manual paso a paso

7.1 Instancia

Se usa una ruta didáctica derivada de RTI-08 (CRISTO BLANCO S.A., 1.57 h por vuelta). Para poder calcular a mano, la duración base se redondea a **90 min**. Hay 3 buses con almuerzo de 2 h (B1 inicia entre 10:30 y 11:30, B2 entre 12:00 y 13:00 y B3 entre 13:30 y 14:30), 8 vueltas con ventanas de la Opción C y el perfil **T2**, que tiene factores redondos.

Vuelta	Ventana de inicio	Duración si sale en r_i	Duración si sale en d_i	Duración en la salida nominal
J1	06:00–06:30	103.7	117.0	110.3
J2	06:30–07:00	117.0	117.0	117.0
J3	07:00–07:30	117.0	110.8	114.2
J4	08:00–08:30	103.8	96.9	100.4
J5	09:30–10:30	90.0	90.0	90.0
J6	11:30–12:30	102.0	108.0	108.0
J7	16:30–17:00	111.0	121.5	116.3
J8	17:00–17:30	121.5	121.5	121.5

Tabla 16. Vueltas de la instancia manual y su duración (min) con tráfico T2.

7.2 Cálculo a mano de la duración con tráfico

J1 saliendo a las 06:00. La franja 06:00–06:30 ($f = 0.9$) dura 30 min reales y cubre $30/0.9 = 33.33$ min base. Faltan $90 - 33.33 = 56.67$ min base, que en la punta mañana ($f = 1.3$) toman $56.67 \times 1.3 = 73.67$ min reales. Duración: $30 + 73.67 = \mathbf{103.67 \text{ min}}$; llega a las 07:43.7.

J4 saliendo a las 08:00. Hasta las 09:00 hay 60 min reales a $f = 1.3$, que cubren 46.15 min base. Los 43.85 restantes se recorren en el valle ($f = 1.0$). Duración: $60 + 43.85 = \mathbf{103.85 \text{ min}}$.

J3 saliendo a las 07:00 frente a las 07:29. Si sale a las 07:00, recorre toda la vuelta en la punta: $90 \times 1.3 = \mathbf{117.0 \text{ min}}$. Si sale a las 07:29, los 91 min reales hasta las 09:00 cubren 70 min base y los 20 restantes van a $f = 1.0$: $91 + 20 = \mathbf{111.0 \text{ min}}$. Salir 29 min más tarde ahorra 6 min de conducción. CP-SAT aprovecha exactamente esto.

Verificación cruzada. La hoja *Calculadora vuelta* del Excel reproduce este cálculo con fórmulas (salida 06:00, base 90, T2 $\rightarrow$ 103.67 min) y coincide con `sched.traffic.travel_time(360, 90)`. Esa coincidencia verifica el modelo de forma cruzada.

7.3 List Scheduling (orden por inicio de ventana)

Paso	Vuelta	B1 (salida–llegada · carga previa)	B2	B3	Elegido
1	J1	06:00–07:44 · 0	06:00–07:44 · 0	06:00–07:44 · 0	B1 (empate → índice)
2	J2	infactible (ocupado)	06:30–08:27 · 0	06:30–08:27 · 0	B2
3	J3	infactible	infactible	07:00–08:57 · 0	B3
4	J4	08:00–09:44 · 103.7	08:27–10:05 · 117.0	infactible (libre 08:57 > 08:30)	B1 (menor carga)
5	J5	09:44–11:14 · 207.5	09:30–11:00 · 117.0	09:30–11:00 · 117.0	B2 (empate → índice)
6	J6	12:30–14:18 · 207.5 (*)	infactible (almuerzo)	11:30–13:12 · 117.0	B3
7	J7	16:30–18:21 · 207.5	16:30–18:21 · 207.0	16:30–18:21 · 219.0	B2 (207.0 < 207.5)
8	J8	17:00–19:02 · 207.5	infactible (J7)	17:00–19:02 · 219.0	B1

Tabla 17. Traza de List Scheduling con tráfico T2. (*) B1 debe almorzar entre 10:30 y 11:30, por eso J6 solo cabe después del almuerzo.

Resultado LS: $B1 = J1 + J4 + J8 = 103.7 + 103.8 + 121.5 = \mathbf{329.0 \text{ min}}$; $B2 = 117.0 + 90.0 + 111.0 = 318.0$; $B3 = 117.0 + 102.0 = 219.0$. $L_{\max} = \mathbf{329.0 \text{ min (5.48 h)}}$.

7.4 LPT adaptado

Las duraciones en la salida nominal (última columna de la tabla de la sección 7.1) dan el orden **J8 (121.5)**, **J2 (117.0)**, **J7 (116.3)**, **J3 (114.2)**, **J1 (110.3)**, **J6 (108.0)**, **J4 (100.4)**, **J5 (90.0)**.

Paso	Vuelta	Elegido	Salida–llegada	Duración	Motivo
1	J8	B1	17:00–19:02	121.5	todos vacíos → índice
2	J2	B2	06:30–08:27	117.0	B2 y B3 con carga 0
3	J7	B3	16:30–18:21	111.0	B1 infactible (J8); B3 carga 0
4	J3	B3	07:00–08:57	117.0	B3 (111) < B1 (121.5); B2 ocupado
5	J1	B1	06:00–07:44	103.7	único factible
6	J6	B1	12:30–14:18	108.0	B1 (225.2) < B3 (228.0)
7	J4	B2	08:27–10:05	97.6	B2 (117) es el de menor carga
8	J5	B2	10:05–11:35	90.0	B1 infactible; B2 (214.6) < B3 (228)

Tabla 18. Traza de LPT con tráfico T2.

Resultado LPT: $B1 = 121.5 + 103.7 + 108.0 = \mathbf{333.2 \text{ min}}$, $B2 = 304.6$ y $B3 = 228.0$. $L_{\max} = \mathbf{333.2 \text{ min}}$, peor que LS. Al colocar primero las vueltas largas de la tarde, LPT reparte las puntas sin mirar la secuencia temporal. Al final J6 solo cabe en B1 después de su almuerzo y queda a una vuelta corta del equilibrio.

7.5 Referencias exactas y comparación

Método	B1	B2	B3	$L_{\max}$ (min)	Brecha vs CP-SAT	Brecha vs cota
Cota inferior LB	—	—	—	284.3	—	0 %
LS	329.0	318.0	219.0	329.0	+5.4 %	+15.7 %
LPT	333.2	304.6	228.0	333.2	+6.8 %	+17.2 %
B&B (337 nodos, 187 podas, 4 ms)	315.2	228.0	316.6	316.6	+1.5 %	+11.4 %
CP-SAT (óptimo, 0.14 s)	232.5	311.6	312.0	312.0	0 %	+9.7 %
T0 (sin tráfico): LS = LPT = B&B = CP-SAT				270.0	0 %	LB = 240

Tabla 19. Comparación en la instancia manual (cargas en minutos).

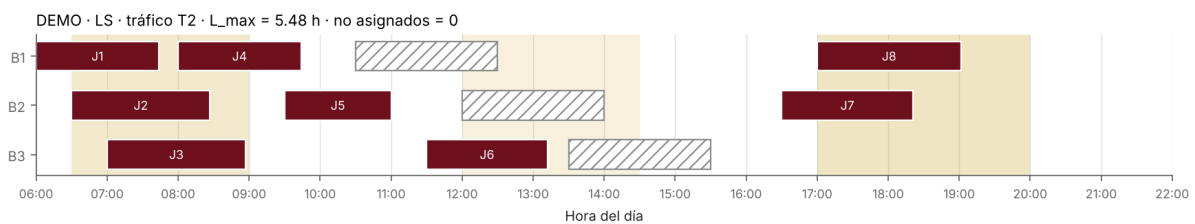


Figura 3. Gantt de List Scheduling (T2). Fondo dorado: franjas de congestión; rayado: almuerzo.

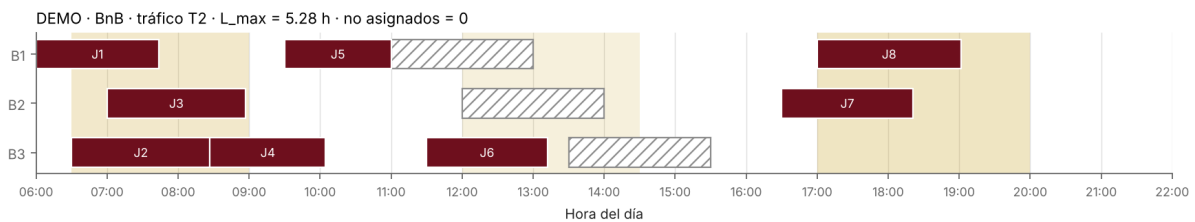


Figura 4. Gantt de la solución de Ramificación y Poda (T2).

Lectura del ejemplo. (1) Sin tráfico todas las estrategias empatan en 270 min (3 + 3 + 2 vueltas de 90 min), y el problema se reduce al del Avance 1. (2) Con tráfico, el **orden** importa y LPT pierde su ventaja clásica. (3) Pasar de LS al B&B mejora 3.8 % solo por reasignar. (4) CP-SAT mejora otro 1.5 % al **retrasar J3 a las 07:29** para salir antes de la punta, lo que muestra que la hora de salida es una decisión con valor propio. Este hallazgo motivó la política `min_duration` evaluada en E1.

8. Estrategia de pruebas y validación

8.1 Validador independiente

validator.py no usa las estructuras internas de los algoritmos. A partir de la instancia y el perfil de tráfico recalcula y verifica: (1) que cada vuelta aparezca exactamente una vez; (2) que el bus pertenezca a la ruta; (3) la ventana; (4) la jornada; (5) que la duración coincida con $\text{travel_time} \times (1 + h)$; (6) que el almuerzo sea único, de 120 min y dentro de su ventana; y (7) que no haya solapamientos. Cada run de la CLI lo ejecuta y termina con error si hay violaciones. En los experimentos se validaron **todas** las soluciones: 0 violaciones en E1 y E2.

8.2 Casos de prueba automatizados (pytest)

Tipo	Prueba	Qué verifica
Básico	test_instancia_manual_valida_y_completa (x9)	LS, LPT y B&B con T0/T1/T2: válidas y completas
Básico	test_resultados_conocidos_ejemplo_manual	270 / 329.0 / 333.2 / 316.6 min (calculados a mano)
Básico	test_calculo_manual_t2	J1 a las 06:00 = 30 + 56.67 × 1.3 min
Propiedad	test_fifo_salir_mas_tarde_nunca_llega_antes	FIFO en 4 000 pares aleatorios
Propiedad	test_bnb_nunca_peor_que_heuristicas	$(U, L_{\max})$ de B&B $\leq$ LS y LPT en 16 instancias
Propiedad	test_cota_inferior_es_valida	$LB \leq L_{\max}$ óptimo
Límite	test_ventana_de_ancho_cero	$r_i = d_i$
Límite	test_vuelta_que_termina_exactamente_a_las_22	Fin = 22:00 es factible
Límite	test_un_solo_bus	$m = 1$
Límite	test_duraciones_iguales_lpt_equivale_a_ls_sin_trafico	LPT degenera en LS
Adverso	test_instancia_infactible_reporta_no_asignados	3 vueltas simultáneas, 1 bus → 2 no asignadas
Adverso	test_vuelta_que_choca_con_el_almuerzo_fijo	Almuerzo fijo bloquea la vuelta
Adverso	test_almuerzo_flexible_se_mueve...	El almuerzo se corre para dejar pasar la vuelta
Adverso	test_lpt_peor_caso_clasico_sin_ventanas	Instancia de Graham {3,3,2,2,2}: LPT = 7, óptimo = 6
Adverso	test_validador_detecta_errores	Una solución corrompida es rechazada
Escala	test_escalas_ruta_grande_es_valida	400 vueltas, 60 buses
Integración	test_excel_genera_37_rutas_y_corrige_rtu28_rtu29	Lectura del Excel y corrección de datos
Integración	test_cpsat_coincide_o_mejora_bnb	CP-SAT óptimo $\leq$ B&B (T0 y T2)
Integración	test_simulacion_*, test_holgura_reduce_retrasos	$\sigma = 0$ reproduce el plan; semilla reproducible; holgura

Tabla 20. Resumen de las 35 pruebas automatizadas (todas aprobadas).

8.3 ¿Cómo se verifica que una salida es correcta?

1. **Factibilidad:** el validador recalcula todas las restricciones con el modelo de tráfico.
2. **Optimalidad en instancias pequeñas:** se compara con el B&B y con CP-SAT; dos métodos exactos independientes que coinciden en 43 de 45 casos sin tráfico. En los 2 restantes el B&B es mejor (−1.8 % y −0.7 %) porque CP-SAT redondea las duraciones hacia arriba al minuto, lo que vuelve infactibles algunas combinaciones muy ajustadas: es un efecto conocido de la discretización.
3. **Calidad en instancias grandes:** brecha respecto de la cota inferior LB.
4. **Cálculo a mano:** la instancia manual (sección 7) y la calculadora del Excel coinciden con el programa.
5. **Instancia nueva propuesta en la sesión:** `python -m sched run --instance nueva.json --alg bnb` resuelve y valida cualquier instancia escrita en el formato de la sección 6.3.

9. Metodología experimental

9.1 Preguntas de evaluación e hipótesis

#	Pregunta	Hipótesis
Q1	¿Qué estrategia atiende más vueltas y logra menor $L_{\max}$ con y sin tráfico?	H1: sin tráfico, $LS \approx LPT$ (duraciones iguales por ruta); con tráfico, LPT pierde ventaja porque ordena por duración e ignora el tiempo.
Q2	¿Cuánto empeora la factibilidad al introducir tráfico?	H2: T1 aumenta poco las vueltas no atendidas; T2 las aumenta de forma marcada en rutas con p alto.
Q3	¿Cuánto vale elegir la hora de salida ($min_duration$)?	H3: reduce el sobre costo por tráfico y $L_{\max}$, a costa de algo de factibilidad.
Q4	¿Qué tan cerca están las heurísticas del óptimo?	H4: brecha media $< 15\%$ frente a CP-SAT; el B&B cronológico queda cerca del óptimo completo.
Q5	¿Cómo escala el tiempo de ejecución?	H5: crecimiento $\approx$ cuadrático con $m \propto n$; milisegundos por ruta real.
Q6	¿Qué tan robusto es el plan ante tráfico variable y cuánto ayuda la holgura?	H6: $\sigma = 0.2$ produce $> 5\%$ de vueltas fuera de ventana; una holgura del 10% lo reduce a costa de atender menos vueltas.

Tabla 21. Preguntas de evaluación e hipótesis.

9.2 Diseño de los experimentos

Exp.	Instancias	Factores	Métricas	Réplicas
E1	37 rutas reales (5 017 vueltas)	Algoritmo {LS, LPT} $\times$ tráfico {T0, T1, T2} $\times$ política {earliest, min_duration}	No asignadas, $L_{\max}$, brecha vs LB, desbalance, sobre costo, tiempo	Determinista
E2	90 aleatorias: $n \in \{8, 10, 12\}$, $m = 3, 15$ semillas, duración base $\pm 25\%$	Algoritmo {LS, LPT, B&B, CP-SAT} $\times$ {T0, T2}	Brecha vs CP-SAT, óptimos alcanzados, factibilidad, tiempo	15 semillas
E3	Sintéticas $n \in \{100 \dots 4\,000\}$, $m = n/3$	Algoritmo {LS, LPT}, T1	Tiempo (mejor de 3)	3
E4	37 rutas reales	Algoritmo $\times$ holgura {0, 10 %} $\times$ σ {0.10, 0.20}, T1	% fuera de ventana (media, p95), retraso, horas extra	100 por ruta

Tabla 22. Diseño experimental.

9.3 Entorno y procedimiento reproducible

- Hardware: Intel Xeon 2.10 GHz (2 núcleos), Linux. Software: Python 3.13, pandas 3.0, matplotlib 3.11, openpyxl 3.1, OR-Tools 9.15.
- Procedimiento: `python -m sched build` y luego `python experiments/run_experiments.py`, que genera `results/E1...E4*.csv` y las figuras en unos 3–4 min. `python experiments/build_excel.py` regenera el Excel.
- Semillas fijas: generador de instancias (seed explícita), simulación (`seed = 2026`) y CP-SAT (`random_seed = 2026`).
- Los tiempos de E3 son el mejor de 3 ejecuciones, para reducir el ruido del sistema.

10. Prototipo inicial y resultados preliminares

10.1 Repositorio

Enlace al repositorio: [\[completar con la URL de GitHub del grupo\]](#)

El repositorio `transporte-cusco-scheduling` contiene el código, las pruebas, los datos, los experimentos y la documentación. Su historial Git permite rastrear la evolución y las contribuciones de cada integrante (Anexo B).

```
transporte-cusco-scheduling/
├── README.md · pyproject.toml · requirements.txt
├── src/sched/          traffic · model · builder · validator · metrics · simulation
│   ├── viz · cli
│   │   └── algorithms/    greedy (LS, LPT) · bnb · cpsat
├── tests/              test_traffic · test_algorithms · test_integration    (35
pruebas)
├── experiments/        run_experiments.py · build_excel.py · demo_prototipo.ipynb
├── data/raw/            Excel fuente (37 empresas, Opción C)
├── data/instances/      37 rutas + DEMO en JSON
├── data/traffic/         perfil_T0/T1/T2.json
├── results/             CSV E1-E4 · figuras · Excel actualizado
└── docs/                plan_aprendizaje · bitacora/ (4) · matriz_contribuciones
```

10.2 Funcionamiento del prototipo

El prototipo cumple con lo pedido en el Avance 3: **carga una instancia, ejecuta al menos un algoritmo y produce una salida verificable**. La demostración en la sesión se hace directamente sobre el sistema:

```
$ python -m sched build
37 instancias + DEMO escritas en data/instances; perfiles en data/traffic

$ python -m sched run --route RTI-01 --alg ls --traffic T1 --gantt
RTI-01 (E.T. SAYLLA S.A.) · 35 buses · 103 vueltas · algoritmo LS · tráfico T1
  L_max = 7.772 h | L_min = 5.207 h | desbalance = 2.565 h | cota inf. = 7.063 h | gap
= 10.0 %
  asignados = 103/103 | utilización = 52.5 % | sobrecosto tráfico = 0.7 % | tiempo =
20.0 ms
  VALIDACIÓN: OK (0 violaciones)
  asignaciones -> results/RTI-01_LS_T1.csv

$ python -m sched demo --traffic T2          # instancia manual: LS, LPT, B&B y CP-SAT
$ python -m sched simulate --route RTI-01 --alg ls --traffic T1 --sigma 0.2
$ python -m pytest -q
35 passed
```

Para validar con una **instancia nueva propuesta durante la sesión**, basta con escribir un JSON con el formato de la sección 6.3 (o editar `data/instances/DEMO.json`) y ejecutar `python -m sched run --instance nueva.json --alg bnb`. El validador confirma la factibilidad y el B&B o CP-SAT dan el óptimo de referencia.

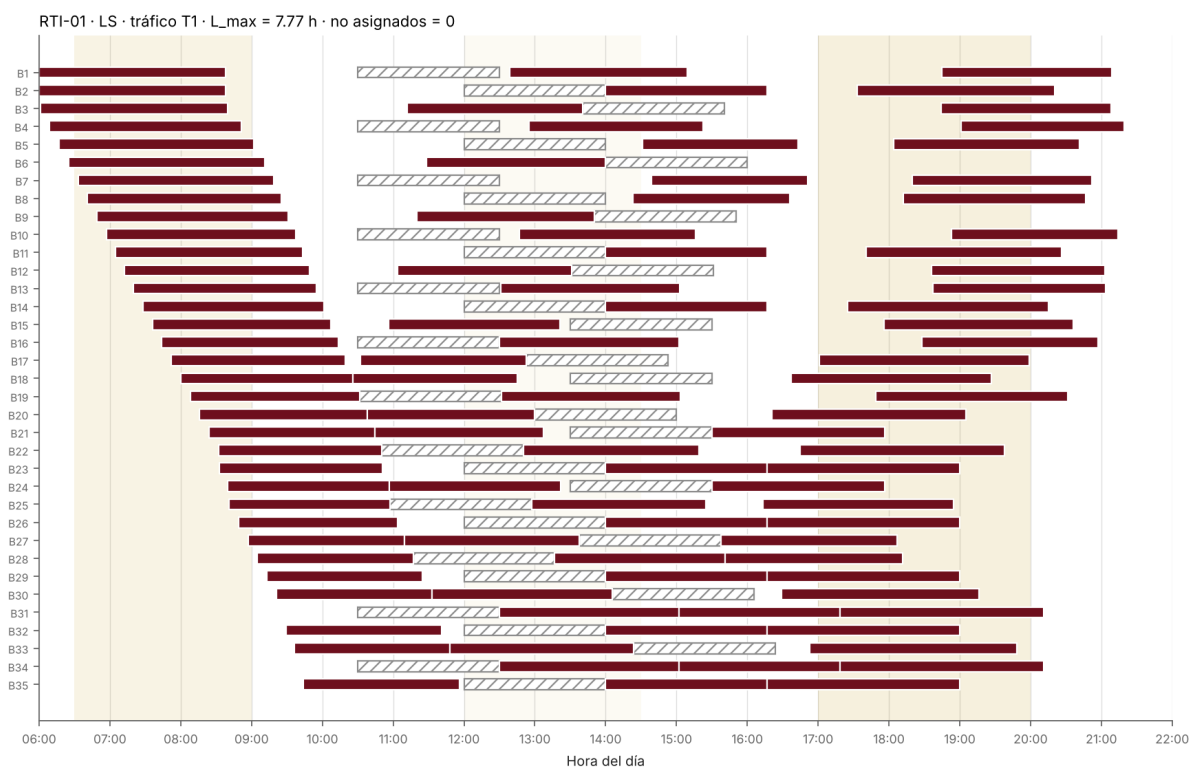


Figura 5. Programa de RTI-01 (35 buses, 103 vueltas) con List Scheduling y tráfico T1.

10.3 E1 — Las 37 rutas bajo tráfico

Tráfico	Política	Alg.	No asignadas	Rutas completas	$L_{\max}$ medio (h)	Brecha vs LB	Desbalance (h)	Sobrecosto tráfico
T0	earliest	LS = LPT	95 (1.9 %)	18	9.23	10.3 %	1.92	0 %
T1	earliest	LS	167	8	9.05	13.7 %	1.68	+0.8 %
T1	earliest	LPT	331	10	9.22	19.9 %	2.61	+0.8 %
T1	min_duration	LS	190	7	8.78	11.0 %	1.67	-2.2 %
T1	min_duration	LPT	373	6	9.12	19.9 %	2.80	-2.2 %
T2	earliest	LS	280	4	9.97	14.5 %	1.94	+13.3 %
T2	earliest	LPT	603	4	10.08	24.1 %	3.47	+13.7 %
T2	min_duration	LS	323	4	9.71	12.5 %	1.96	+10.3 %
T2	min_duration	LPT	605	4	9.79	20.4 %	3.42	+10.4 %

Tabla 23. Resultados agregados de E1 (5 017 vueltas; brecha y desbalance promediados por ruta; 0 violaciones en todas las configuraciones).

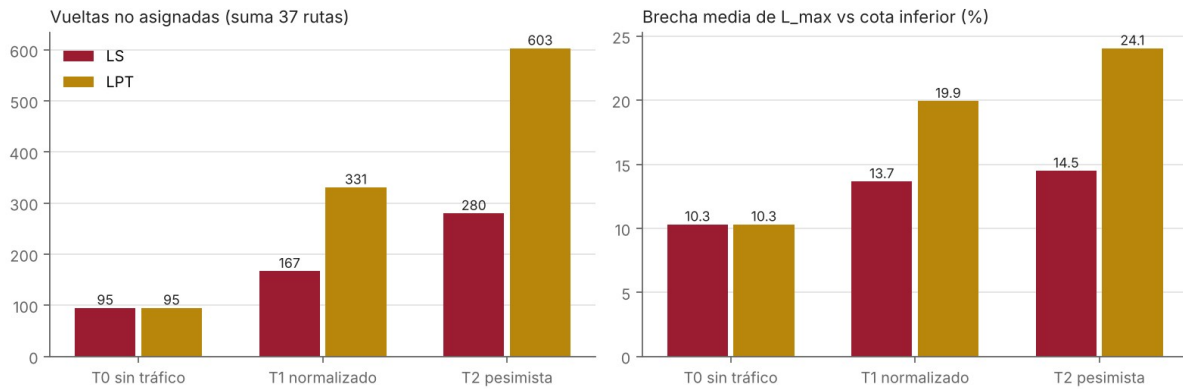


Figura 6. Vueltas no asignadas y brecha media respecto de la cota inferior, por escenario de tráfico.

- **Sin tráfico, LS y LPT coinciden exactamente.** Dentro de cada ruta todas las vueltas duran lo mismo, así que el orden de LPT empata y se desempata por r_j , que es el orden de LS. Es el caso degenerado previsto en el Avance 1.
- **Con tráfico, LS supera a LPT en factibilidad y en equilibrio.** En T1 deja la mitad de vueltas sin atender (167 frente a 331) y es mejor o igual en 28 de 37 rutas. En T2 es mejor en 33 de 37 rutas. El desbalance de LPT es 55–80 % mayor.
- **El tráfico pesimista reduce la atención** de 98.1 % (T0) a 96.7 % (T1) y 94.4 % (T2) con LS. En T1 el sobrecosto total es pequeño (+0.8 %) porque el perfil solo redistribuye el tiempo; en T2 es +13.3 %.
- **Elegir la hora de salida tiene un efecto doble.** Con `min_duration`, LS reduce $L_{\max}$ de 9.05 a 8.78 h y la brecha de 13.7 % a 11.0 %; el sobrecosto pasa a -2.2 %, es decir, las vueltas evitan las puntas. A cambio deja 23 vueltas más sin atender, porque salir más tarde consume holgura de la ventana.
- **Las vueltas no atendidas sin tráfico se concentran en rutas saturadas:** RTU-22 (21; $p = 1.02$), RTI-02 (15; $p = 0.87$), RTU-01 (12; $p = 0.79$) y RTU-05 (7; $p = 0.82$). La saturación p anticipa la dificultad.
- **Rendimiento:** cada configuración resuelve las 37 rutas en unos 0.6 s (≈ 16 ms por ruta) con la política `earliest` y en unos 3 s con `min_duration`.

10.4 E2 — Heurísticas frente a métodos exactos

Tráfico	Algoritmo	Brecha media vs CP-SAT	Brecha máx.	Óptimo alcanzado	Peor factibilidad que el óptimo	Tiempo medio
T0	LS	8.1 %	44.4 %	4 / 45	9 / 45	0.19 ms
T0	LPT	8.7 %	35.3 %	2 / 45	26 / 45	0.15 ms
T0	B&B	-0.07 %	0 %	43 / 45	0 / 45	21 ms
T0	CP-SAT	0	0	45 / 45	—	44 ms
T2	LS	10.5 %	34.9 %	0 / 45	12 / 45	0.21 ms
T2	LPT	17.6 %	59.6 %	0 / 45	32 / 45	0.17 ms
T2	B&B	2.1 %	6.3 %	8 / 45	0 / 45	23 ms
T2	CP-SAT	0	0	45 / 45	—	294 ms

Tabla 24. E2: 45 instancias aleatorias por escenario ($n \in \{8, 10, 12\}$, $m = 3$). Las brechas solo se promedian donde el número de vueltas atendidas coincide con el óptimo. CP-SAT terminó con estado OPTIMAL en las 90.

Las heurísticas resuelven en fracciones de milisegundo, con una brecha media de 8–10 % (LS) y una peor factibilidad en el 20–27 % de los casos. LPT pierde factibilidad en más de la mitad de las instancias. Con tráfico, el B&B (salida más temprana) queda a 2.1 % del óptimo completo: **ese 2 % es el valor de decidir la hora de salida**, y coincide con lo observado en la instancia manual (1.5 %).

10.5 E3 — Escalabilidad

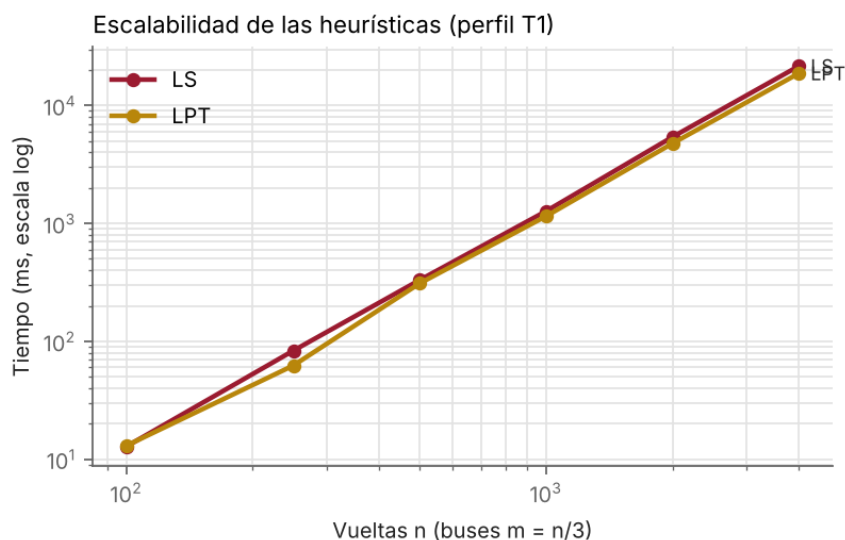


Figura 7. Tiempo de ejecución de LS y LPT frente al número de vueltas (escala logarítmica).

La pendiente en escala log-log es ≈ 2 : duplicar n multiplica el tiempo por unas 4, lo que concuerda con el análisis $O(n \cdot m \cdot k)$ con $m \propto n$. Una instancia de 4 000 vueltas y 1 333 buses (unas 25 veces la ruta más grande) se resuelve en 22 s. El índice por disponibilidad de buses (sección 11.4) reduciría este costo si se necesitara.

10.6 E4 — Robustez ante la variabilidad del tráfico

Algoritmo	Holgura	σ	% fuera de ventana (media / p95)	Retraso medio (min)	% buses con horas extra	No asignadas en el plan
LS	0 %	0.10	6.0 / 9.2	1.7	0.2	167
LS	0 %	0.20	8.8 / 12.7	4.1	1.7	167
LS	10 %	0.10	1.7 / 3.7	0.4	0.1	236
LS	10 %	0.20	5.6 / 9.2	2.2	1.6	236
LPT	0 %	0.10	2.6 / 4.6	1.2	0.2	331
LPT	0 %	0.20	5.0 / 8.2	3.2	2.0	331
LPT	10 %	0.10	0.6 / 1.8	0.2	0.1	530
LPT	10 %	0.20	2.6 / 5.1	1.5	1.5	530

Tabla 25. E4: simulación Monte Carlo con tráfico T1, 100 réplicas por ruta (37 rutas).

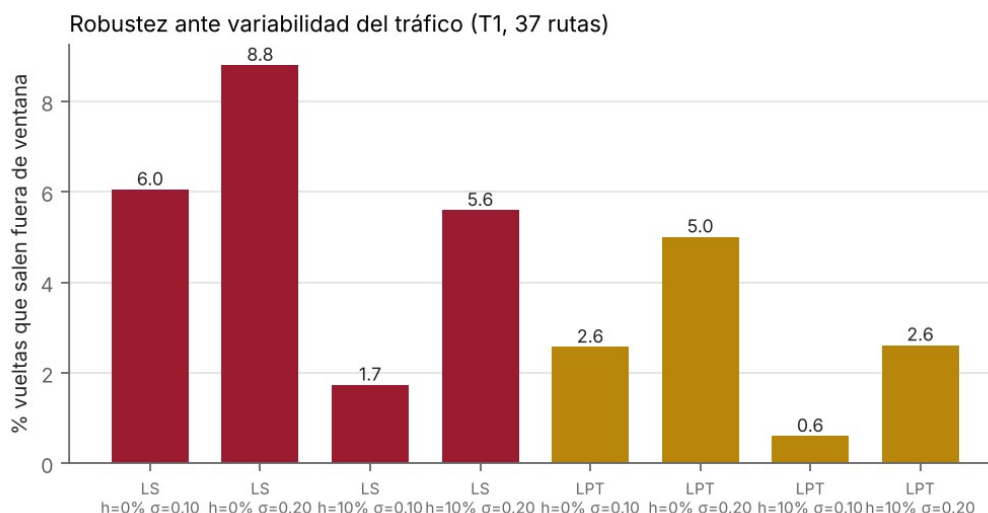


Figura 8. Porcentaje de vueltas que salen fuera de su ventana en la ejecución simulada.

Un plan sin holgura y con variabilidad alta ($\sigma = 0.20$) produce un 8.8 % de salidas fuera de ventana con LS. Una holgura del 10 % lo reduce a 5.6 %, pero deja 69 vueltas más sin atender. LPT parece más robusto, pero esa ventaja es engañosa: atiende muchas menos vueltas y por eso sus buses tienen más tiempo libre. La comparación de robustez debe hacerse **a igual número de vueltas atendidas**, y esto se corregirá en la Entrega 2 (sección 11.5).

10.7 Contraste con las hipótesis

Hipótesis	Resultado	Estado
H1: LS $\approx$ LPT sin tráfico; LPT pierde con tráfico	Idénticos en T0; LPT peor en T1 y T2	Confirmada
H2: T1 afecta poco; T2 mucho	+72 vueltas no atendidas en T1 (+1.4 pp) y +185 en T2 (+3.7 pp) respecto de T0	Confirmada
H3: min_duration reduce sobre costo y $L_{\max}$ a costa de factibilidad	-3 pp de sobre costo, -0.27 h de $L_{\max}$, +23 no asignadas (T1)	Confirmada
H4: brecha media < 15 % vs óptimo	LS 8-10 % ✓; LPT 17.6 % en T2 ×; B&B 2.1 % ✓	Parcial
H5: crecimiento $\approx$ cuadrático	Pendiente log-log $\approx$ 2	Confirmada
H6: $\sigma = 0.2$ da > 5 % fuera de ventana; la holgura ayuda con costo	8.8 % $\rightarrow$ 5.6 % con 10 % de holgura; +69 no asignadas	Confirmada

Tabla 26. Contraste de hipótesis (resultados preliminares).

11. Discusión, ajustes y dificultades

11.1 Interpretación y contraste con la teoría

¿Por qué LPT pierde con ventanas y tráfico? La garantía de Graham para LPT supone que cualquier trabajo puede ir en cualquier momento, de modo que solo importa el tamaño. Con ventanas, el orden de procesamiento determina la factibilidad. Cuando las ventanas son estrechas, el problema se parece a la partición de intervalos, donde el algoritmo voraz que procesa los intervalos **por hora de inicio** es óptimo en número de máquinas (Kolen et al., 2007). LS conserva ese orden. LPT lo rompe: inserta primero vueltas de la tarde y deja huecos fragmentados en la mañana.

Con tráfico, además, la "vuelta más larga" es la que cae en hora punta. LPT termina ordenando por congestión y no por tamaño real, y agrupa las vueltas de punta en los mismos buses. El aumento del desbalance (1.68 → 2.61 h en T1) lo confirma. Es un buen ejemplo de que una garantía teórica no se traslada automáticamente cuando cambian los supuestos del modelo.

Alternativa descartada: best fit. Elegir el bus que deja menos tiempo muerto antes de la vuelta es una regla natural en la partición de intervalos, así que se probó. Resultó peor en todos los escenarios: T1 deja 177 vueltas sin atender (frente a 167) y eleva el $L_{\max}$ medio a 12.7 h (frente a 9.0). Al compactar las líneas de tiempo, concentra las vueltas en pocos buses y choca con sus almuerzos. La regla de menor carga reparte mejor.

El valor de la hora de salida. CP-SAT y la política `min_duration` muestran que decidir *cuándo* sale una vuelta, no solo *en qué bus*, reduce la carga en 1.5–3 %. Este grado de libertad no existía en el Avance 1 y aparece precisamente por modelar el tráfico.

11.2 Cuándo conviene cada alternativa

Alternativa	Conviene cuando...	Limitación
LS (earliest)	Se busca atender el máximo de vueltas con rapidez (operación diaria).	No aprovecha la hora de salida; brecha de 8–14 %.
LS (min_duration)	El objetivo principal es reducir horas de conducción y consumo en días de tráfico.	Atiende algo menos de vueltas.
LPT	No hay ventanas o son muy anchas y las duraciones varían (caso clásico).	Con ventanas y tráfico pierde factibilidad y equilibrio.
B&B	Instancias de hasta unas 12 vueltas; validar heurísticas; enseñanza.	Exponencial; solo explora salidas tempranas.
CP-SAT	Referencia óptima en instancias pequeñas y medianas.	Discretización a 1 min; en rutas completas no prueba optimalidad en 2 min.
Holgura del 10 %	Variabilidad alta y prioridad a la puntualidad.	Reduce las vueltas atendidas.

Tabla 27. Situaciones en que conviene cada alternativa.

11.3 Ajustes realizados a partir de los avances anteriores

Observación o problema detectado	Ajuste en la Entrega 1
Observación del docente: ¿cómo se maneja el tráfico?	Duración dependiente de la hora (capa A, FIFO), escenarios T0/T1/T2 y variabilidad simulada (capa B).
Avance 1: se minimizaba $C_{\max}$ pero se reportaba la carga máxima.	Objetivo corregido a $L_{\max}$ y criterio lexicográfico con vueltas atendidas.
Avance 1: almuerzo declarado pero ausente en el ejemplo; almuerzo fijo.	Almuerzo con ventana de ± 30 min en 3 turnos. Con almuerzo fijo, RTI-01 perdía 4 vueltas incluso sin tráfico, una infactibilidad artificial.
Avance 1: "Branch and Bound" sin especificar.	B&B implementado con cota, simetría e incumbente, y CP-SAT como verificación cruzada.
Avance 2: faltaban módulos, formatos, pseudocódigo, complejidad y pruebas.	Secciones 5, 6 y 8; 35 pruebas automatizadas; validador independiente.
Avance 3: se pedía repositorio y prototipo ejecutable.	Paquete sched con CLI, README, cuaderno de demostración y experimentos reproducibles.
Datos: RTU-28 y RTU-29 con 3 057 y 2 836 vueltas.	Detección automática y regeneración (145 y 139); marcado en el Excel.
Algunas rutas no pueden atender todas las vueltas.	Indicador de saturación p y reporte explícito de vueltas no asignadas.

Tabla 28. Ajustes respecto de los avances 1, 2 y 3.

11.4 Dificultades pendientes

- **Mejorar las heurísticas.** Una búsqueda local (reubicar o intercambiar vueltas entre buses) partiendo de LS podría cerrar parte de la brecha de 8–14 %. Es la tercera estrategia prevista para la Entrega 2.
- **Combinar LPT con el orden temporal.** Una variante de LPT por franja (ordenar por duración dentro de cada periodo) podría conservar la idea de LPT sin romper el orden temporal.
- **Calibrar el tráfico.** Los factores son supuestos. Se buscará medir tiempos reales de algunas vueltas o conseguir datos abiertos de GPS para calibrar al menos una ruta.
- **Robustez a igual servicio.** Comparar E4 con el mismo conjunto de vueltas atendidas (por ejemplo, holgura solo donde sobra capacidad).
- **Escala de los métodos exactos.** CP-SAT no prueba optimalidad en rutas completas en 2 min. Se evaluarán cotas más fuertes o la descomposición por franjas.
- **Eficiencia.** Un índice de buses por hora de disponibilidad evitaría recorrer los m buses en cada vuelta.

11.5 Amenazas a la validez

Tipo	Amenaza	Mitigación
Constructo	Las ventanas, los turnos de almuerzo y los factores de tráfico son supuestos, no observaciones.	Se declaran explícitamente, son configurables y se evalúan en varios escenarios (T0/T1/T2).
Constructo	$L_{\max}$ no captura todo el costo operativo (consumo, puntualidad).	Métricas complementarias: desbalance, sobrecosto, robustez.
Interna	Comparar algoritmos que atienden distinto número de vueltas sesga $L_{\max}$ y la robustez.	Objetivo lexicográfico; brecha contra la cota de las vueltas atendidas; corrección pendiente en E4.
Interna	La discretización de CP-SAT puede alterar el óptimo.	Validación con duraciones exactas; contraste con el B&B.
Externa	Datos de 2020 y una sola jornada típica.	Los resultados se presentan como comparativos entre algoritmos, no como predicción operativa.
Externa	E2 usa instancias sintéticas pequeñas.	Semillas fijas y tres tamaños; las conclusiones se contrastan con E1 (rutas reales).
Conclusión	Pocas réplicas o tiempos con ruido.	100 réplicas por ruta en E4; mejor de 3 ejecuciones en E3; percentil 95.

Tabla 29. Amenazas a la validez y mitigaciones.

12. Conclusiones y recomendaciones

12.1 Conclusiones

1. **El tráfico se incorporó como un factor explícito y verificable.** La duración de cada vuelta depende de su hora de salida mediante un modelo por franjas que cumple FIFO. Se evalúa con tres escenarios y con simulación estocástica, lo que responde a la observación del docente.
2. **La formulación se corrigió y se volvió más realista.** El objetivo pasó a ser la carga máxima $L_{\max}$ con prioridad a las vueltas atendidas, el almuerzo tiene ventana y la hora de salida es una variable de decisión.
3. **List Scheduling es la mejor heurística en este contexto.** Con tráfico deja sin atender la mitad de vueltas que LPT (167 frente a 331 en T1) y equilibra mejor la carga. La ventaja teórica de LPT no resiste las ventanas ni el tráfico.
4. **El tráfico tiene un costo medible.** Con el perfil pesimista la atención cae de 98.1 % a 94.4 % y la conducción total sube 13 %. Elegir la hora de salida recupera cerca de 3 puntos de sobre costo.
5. **Los métodos exactos confirman la calidad de las heurísticas y cuantifican la mejora posible.** LS queda en promedio a 8–10 % del óptimo; decidir la hora de salida vale cerca de 2 %.
6. **La evaluación crítica de los datos cambió los resultados.** Corregir RTU-28/29 eliminó el 55 % de vueltas espurias, y la saturación ρ mostró que RTU-22 es infactible por capacidad, no por el algoritmo.
7. **El prototipo es reproducible:** 37 instancias, 4 algoritmos, validador independiente, 35 pruebas, experimentos con semillas y un Excel con 536 fórmulas que replica el modelo de tráfico.

12.2 Lecciones aprendidas

- Una garantía teórica (la de LPT) depende de sus supuestos, y hay que verificarla experimentalmente cuando el modelo cambia.
- Un validador independiente y una instancia resuelta a mano detectaron errores que las métricas agregadas ocultaban.
- Antes de culpar al algoritmo conviene revisar los datos: la mitad de las vueltas no atendidas venía de un error de la fuente.

12.3 Trabajo para la Entrega 2

Actividad	Responsable	Producto
Búsqueda local sobre LS (tercera estrategia) y LPT por franja	Choque, Aguilar	algorithms/ local_search.py
Cotas más fuertes y B&B con salida flexible	Huacani	bnb.py v2
Calibración del perfil de tráfico de una ruta	Moreano, Aguilar	data/traffic/ perfil_calibrado.json
Robustez a igual servicio y análisis estadístico (Wilcoxon)	Moreano	E4 v2
Pruebas de integración ampliadas y documentación técnica	Todos	tests/, manual técnico
Actualización de bitácoras individuales	Cada integrante	docs/bitacora/

Tabla 30. Plan para la Entrega 2.

13. Referencias

- Ceder, A. (2016). *Public transit planning and operation: Modeling, practice and behavior* (2.^a ed.). CRC Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Garey, M. R. y Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.
- Gawiejnowicz, S. (2008). *Time-dependent scheduling*. Springer.
- Google. (2024). *OR-Tools: CP-SAT solver* [Documentación oficial].
<https://developers.google.com/optimization>
- Graham, R. L. (1966). Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45(9), 1563–1581.
- Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2), 416–429.
- Ibarra-Rojas, O. J., Delgado, F., Giesen, R. y Muñoz, J. C. (2015). Planning, operation, and control of bus transport systems: A literature review. *Transportation Research Part B: Methodological*, 77, 38–75.
- Ichoua, S., Gendreau, M. y Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2), 379–396.
- Kolen, A. W. J., Lenstra, J. K., Papadimitriou, C. H. y Spieksma, F. C. R. (2007). Interval scheduling: A survey. *Naval Research Logistics*, 54(5), 530–543.
- Land, A. H. y Doig, A. G. (1960). An automatic method of solving discrete programming problems. *Econometrica*, 28(3), 497–520.
- Law, A. M. (2015). *Simulation modeling and analysis* (5.^a ed.). McGraw-Hill.
- Lee, C.-Y. (1996). Machine scheduling with an availability constraint. *Journal of Global Optimization*, 9, 395–416.
- Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P. y Wießner, E. (2018). Microscopic traffic simulation using SUMO. En *21st IEEE International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582).
- Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5.^a ed.). Springer.
- Tabla de datos – Unidades de transporte – Flota operativa 2020. (s. f.). Scribd.
<https://es.scribd.com/document/553897271/TABLA-DE-DATOS-UNIDADES-DE-TRANSPORTE-FLOTA-OPERATIVA-2020>
- Ugarte Rojas, H. E. (2026). *Proyecto semestral: Desarrollo y evaluación experimental de una solución basada en algoritmos avanzados* (v2.0). Escuela Profesional de Ingeniería Informática y de Sistemas, UNSAAC.

Anexo A. Manual de instalación y uso

Requisitos: Python ≥ 3.10 (probado con 3.13), en Windows, Linux o macOS.

```
git clone <URL-del-repositorio> && cd transporte-cusco-scheduling
python -m venv .venv
source .venv/bin/activate                # Windows: .venv\Scripts\activate
pip install -r requirements.txt
pip install -e .

python -m sched build                    # genera data/instances/*.json
python -m sched profiles                 # muestra T0, T1, T2
python -m sched demo --traffic T2       # ejemplo manual
python -m sched run --route RTU-05 --alg lpt --traffic T1 --policy min_duration --
gantt
python -m sched simulate --route RTI-01 --alg ls --traffic T1 --sigma 0.2 --buffer 0.1
python -m pytest -q                     # 35 pruebas
python experiments/run_experiments.py   # E1-E4 (≈ 3–4 min)
python experiments/build_excel.py       # Excel actualizado
```

Opción	Valores	Efecto
--route / --instance	código de ruta, DEMO o archivo JSON	Instancia a resolver
--alg	ls · lpt · bnb · cpsat	Algoritmo
--traffic	T0 · T1 · T2 · archivo.json	Perfil de tráfico
--policy	earliest · min_duration	Política de hora de salida
--buffer	0.10 = 10 %	Holgura de planificación
--sigma, --reps, --seed	p. ej. 0.2, 200, 2026	Parámetros de la simulación
--gantt, --out	—	Figura y ruta del CSV

Tabla 31. Opciones de la línea de comandos.

Sin `pip install -e .` también funciona anteponiendo `PYTHONPATH=src` (Linux/macOS) o ejecutando `set PYTHONPATH=src` (Windows). El cuaderno `experiments/demo_prototipo.ipynb` reproduce la demostración en Google Colab.

Anexo B. Matriz de contribuciones

R = responsable, A = apoyo, V = revisión. La matriz se contrastará con el historial del repositorio (log --author).

Entregable / módulo	Aguilar	Choque	Huacani	Moreano
Reformulación y modelo de tráfico	R	V	V	A
Modelo de datos y línea de tiempo	R	A	V	
List Scheduling y LPT	A	R	V	
Ramificación y poda	V	A	R	
CP-SAT	R		A	
Validador y métricas	A		R	V
Pruebas automatizadas	A	A	R	A
Datos, constructor y Excel	V			R
Simulación y experimentos	A			R
Visualización			V	R
Ejemplo manual (verificación a mano)	A	R	R	V
Monografía	R	A	A	A

Tabla 32. Matriz de contribuciones (propuesta; ajustar a lo realizado).

Anexo C. Formato de bitácora individual

Cada integrante mantiene docs/bitacora/<apellido>.md y registra **una fila por sesión de trabajo en el momento en que ocurre**. Es la evidencia principal del indicador AG-C06.01 y se evalúa de forma individual.

Fecha	Necesidad	Actividad	Fuente	Qué aprendí	Aplicación (archivo/commit)	Dificultad y solución	Horas

Tabla 33. Formato de registro de la bitácora.

Al cierre de cada entrega, cada integrante agrega una reflexión breve: qué sabía antes y qué sabe ahora, qué fuente le resultó más confiable y por qué, y qué necesita aprender para la siguiente etapa.

Anexo D. Correspondencia con los requisitos de la Entrega 1

Requisito	Fuente del requisito	Sección
Formulación, objetivos y alcance	Entrega 1 / Avance 1	1, 4
Entradas, salidas, restricciones, supuestos y función objetivo	Avance 1	4.3–4.7
Manejo del tráfico (observación del docente)	Revisión del docente	1.2, 4.2, 10.3, 10.6
Fundamentos y estado del arte	Entrega 1 / AG-C06.03	2.1–2.2
Necesidades y plan de aprendizaje autónomo	Entrega 1 / AG-C06.01	2.3–2.4, Anexo C
Stack justificado y matriz de comparación	Entrega 1 / Avance 2 / AG-C06.02	3
Módulos, estructuras, flujo y formatos	Avance 2	5.2, 6
Pseudocódigo y análisis de complejidad	Avance 2	5.3–5.5
Instancia pequeña resuelta paso a paso	Avance 1 / Avance 2	7
Pruebas básicas, límite y adversas; verificación de salidas	Avance 2	8
Métricas para comparar algoritmos	Avance 2	4.7, 9
Repositorio activo y su organización	Avance 3	10.1, Anexo A
Prototipo que carga una instancia, ejecuta un algoritmo y produce una salida verificable	Avance 3	10.2
Dificultades pendientes y ajustes respecto de avances anteriores	Avance 3	11.3–11.4
Al menos dos algoritmos y una referencia exacta	Descripción del proyecto §5, §13.4	5
Reproducibilidad (dependencias, semillas, sin rutas absolutas)	Descripción del proyecto §13.7, §13.13	6.4, 9.3

Tabla 34. Correspondencia entre los requisitos y las secciones del informe.

Anexo E. Resultados por ruta

ρ = saturación de la flota; NA = vueltas no asignadas; $L_{\max}$ en horas. Política de salida earliest.
Archivo completo: `results/E1_37_rutas.csv` y hoja *Resultados por ruta* del Excel.

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS $L_{\max}$ T1	LPT NA T1	LPT $L_{\max}$ T1	LS NA T2	LPT NA T2
RTI-01	35	103	0.52	0	0	7.77	3	9.73	4	13
RTI-02	27	127	0.87	15	19	10.74	36	10.70	29	43
RTI-04	42	144	0.55	1	2	8.64	4	9.04	5	16
RTI-05	44	150	0.48	0	1	7.55	0	7.87	2	3
RTI-06	32	144	0.56	0	3	8.53	0	8.67	4	8
RTI-07	39	144	0.64	4	8	9.46	15	9.90	11	28
RTI-08	20	108	0.61	0	1	9.23	1	9.31	3	11
RTI-09	43	135	0.54	0	2	9.21	8	9.54	6	17
RTI-10	37	166	0.62	1	4	9.48	11	9.73	7	19
RTU-01	32	144	0.79	12	15	11.93	28	10.12	22	39
RTU-02	29	117	0.63	2	4	8.91	12	8.95	8	18
RTU-04	28	144	0.75	5	7	10.53	21	10.35	13	29
RTU-05	24	144	0.82	7	10	11.50	28	9.91	18	36
RTU-07	28	127	0.46	0	0	7.03	0	7.11	1	0
RTU-08	26	114	0.55	0	2	8.64	2	8.77	3	6
RTU-09	21	120	0.65	0	1	9.70	5	9.60	4	12
RTU-10	16	103	0.61	0	0	9.11	2	9.26	1	6
RTU-11A	21	103	0.42	0	0	6.22	0	6.13	0	0
RTU-11B	22	103	0.40	0	0	6.06	0	6.16	0	0
RTU-12	39	150	0.52	2	3	7.62	0	7.77	4	5
RTU-13	37	144	0.65	3	8	9.32	16	11.65	11	26
RTU-14	37	108	0.42	0	0	6.35	0	6.21	0	0
RTU-15	30	127	0.58	1	4	9.21	3	9.58	5	11
RTU-16	40	150	0.53	2	2	7.86	0	7.97	5	10
RTU-17	28	127	0.57	0	2	8.72	1	8.95	3	7
RTU-18	35	166	0.58	0	4	8.54	0	10.12	5	8
RTU-19	34	144	0.70	6	10	9.51	21	9.44	14	31
RTU-20	31	154	0.55	0	2	7.87	2	7.86	4	6
RTU-21	26	166	0.59	0	0	8.86	0	9.00	0	6
RTU-22	19	127	1.02	21	25	12.94	39	12.83	37	52
RTU-23	37	166	0.60	1	5	9.23	4	9.39	5	19
RTU-24	35	117	0.58	2	4	9.42	7	9.70	7	22
RTU-25	27	127	0.56	0	2	8.29	1	8.42	3	5
RTU-26	33	154	0.66	2	5	9.79	16	10.00	9	23
RTU-27	26	166	0.65	0	0	9.73	7	9.84	2	12
RTU-28	31	145	0.69	3	6	10.30	18	10.46	11	28
RTU-29	30	139	0.72	5	6	10.87	20	11.15	14	28

Ruta	Buses	Vueltas	ρ	NA T0	LS NA T1	LS $L_{\max}$ T1	LPT NA T1	LPT $L_{\max}$ T1	LS NA T2	LPT NA T2
Total	1141	5017		95	167		331		280	603

Tabla 35. Resultados por ruta (resaltadas: rutas con $\rho \geq 0.85$).